

Universidade da Beira Interior

Department of Computer Science

Nº 1 - 2025/2026: RAT Analyzer — Integrated Static and Dynamic
Analysis
Platform for Remote Access Trojan Detection



Developed by:

José Miguel Gonçalves de Amoreira — 51712

Supervisor:

Professor João Bernardo Ferreira Sequeiros

June 24, 2026

Abstract

This report presents RAT ANALYZER, an integrated static and dynamic analysis platform for detecting *Remote Access Trojans* (RATs) in Windows executables (and C# source with reduced scope), developed as the final-year project in Computer Science Engineering at Universidade da Beira Interior. The solution combines a Python backend (FastAPI) with a seven-phase pipeline (PE metadata, ILSpy, heuristics, deobfuscation, YARA, risk scoring, native/Ghidra when needed, and reporting), a React web interface (with optional Gemini assistant), a WPF desktop application, and an isolated Hyper-V sandbox with two independent orchestration paths (HTTP vm-agent and PowerShell Direct with SHA-256 integrity checks). Validation relies on 220 automated tests, proxy detection metrics on 59 labeled synthetic profiles stratified across five severity levels (precision/recall/F1 and level accuracy 1.00), a benign corpus of eight compiled PEs analysed statically (specificity 1.00, zero false positives), public MalwareBazaar IOC metadata (without host-side malware storage), a formal static benchmark (median 36.1 s, $n = 5$), synthetic scoring profiles aligned with `test_risk_scoring.py`, and benign execution via `benign-vm-test.exe` on Path B. The system was designed for explainability and local isolation rather than commercial cloud sandboxes, targeting academic and laboratory use.

Keywords: malware analysis; RAT; sandbox; Hyper-V; YARA; FastAPI; software engineering; cybersecurity.

Resumo

Este relatório apresenta o RAT ANALYZER, uma plataforma integrada de análise estática e dinâmica para detecção de *Remote Access Trojans* (RATs) em executáveis Windows (e código C# com âmbito reduzido), desenvolvida no âmbito do projeto final de Licenciatura em Engenharia Informática na Universidade da Beira Interior. A solução combina um backend Python (FastAPI) com um pipeline de sete fases (metadados PE, ILSpy, heurísticas, desofuscação, YARA, scoring de risco, nativo/Ghidra quando necessário e relatório), uma interface web React com assistente Gemini opcional, uma aplicação desktop WPF e uma sandbox Hyper-V isolada com dois caminhos de orquestração independentes (vm-agent HTTP e PowerShell Direct com verificação de integridade SHA-256). A validação assenta em 220 testes automatizados, métricas de detecção proxy em 59 perfis sintéticos etiquetados estratificados por cinco níveis de severidade (precisão/recall/F1 e exatidão de nível 1,00), um corpus benigno de oito PEs compilados analisados estaticamente (especificidade 1,00, zero falsos positivos), metadados IOC públicos do MalwareBazaar (sem armazenamento de malware no host), um benchmark estático formal (mediana 36,1 s, $n = 5$), perfis de scoring sintéticos alinhados com `test_risk_scoring.py` e execução benigna via `benign-vm-test.exe` no Caminho B. O sistema foi desenhado para explicabilidade e isolamento local em detrimento de sandboxes comerciais na cloud, destinando-se a uso académico e laboratorial.

Palavras-chave: análise de malware; RAT; sandbox; Hyper-V; YARA; FastAPI; engenharia de software; cibersegurança.

Acknowledgements

I would like to express my sincere gratitude to everyone who played a significant role in the successful completion of this final-year project in Computer Science Engineering. The support, dedication, and expertise of each person were essential to bringing this work to fruition, and I am truly grateful for every contribution received.

First and foremost, I wish to thank my supervisor, **Professor João Bernardo Ferreira Sequeiros**, for his exceptional guidance, constant availability, and rigorous feedback throughout the entire development process. His supervision was decisive in guiding the most challenging technical decisions, validating the malware analysis approaches adopted, and keeping the project aligned with objectives that were simultaneously academic and practical. With his contribution, I was able to grow both as a student and as a professional.

I also acknowledge the invaluable contribution of the **open-source** community: tools such as YARA, Ghidra, ILSpy, FastAPI, React, and many others made it possible to build an integrated malware analysis platform in an academic context, without the financial resources of commercial solutions. The quality and documentation of these tools were an anchor throughout the entire development process.

Finally, I wish to thank my family and friends for their unconditional support and constant encouragement throughout this journey. The emotional stability they provided was crucial to maintaining determination and perseverance during the most demanding moments. Without them, this project would not have been possible.

In summary, this project was a significant learning experience, made possible by the commitment and dedication of everyone who participated in it, directly or indirectly. Thank you all very much.

Contents

Abstract	1
Resumo	2
Acknowledgements	3
Acronyms	10
1 Introduction	11
1.1 Context	11
1.2 Motivation	11
1.3 Objectives	12
1.4 Document Organisation	12
2 State of the Art	14
2.1 Background	14
2.2 Malware Analysis: Overview	14
2.3 Trends in RAT and Malware Detection	14
2.3.1 Commercial and cloud sandboxes	14
2.3.2 VirusTotal	15
2.3.3 Open-source frameworks	15
2.3.4 Machine learning-based detection	15
2.4 Positioning relative to existing solutions	16
2.5 Conclusions	16
3 Technologies and Tools Used	17
3.1 Introduction	17
3.2 Python and FastAPI	17
3.3 React, Vite and TypeScript	17
3.4 .NET 8, WPF and Minimal API	17
3.5 YARA, Ghidra and ILSpy	18
3.6 Hyper-V, PowerShell and SQLite	18
3.7 GitHub and LaTeX	18
3.8 Security and Isolation	19
3.9 Technology choices and rejected alternatives	19
3.10 Conclusion	20
4 Software Engineering	21
4.1 Introduction	21
4.2 Requirements Analysis	21

CONTENTS

4.2.1	Functional Requirements	21
4.2.2	Non-Functional Requirements	25
4.2.3	Requirements traceability	26
4.3	Diagrams	26
4.3.1	Use Case Diagram	26
4.3.2	Activity Diagram - Static Analysis	27
4.3.3	Activity Diagram - Dynamic Analysis	28
4.3.4	System Architecture	29
4.3.5	Trust zones and authentication	31
4.3.6	Sandbox paths (A and B)	33
4.4	Security Architecture	35
4.5	Sandbox Path Comparison	37
4.6	Data Model	37
4.7	Conclusions	38
5	Implementation	40
5.1	Introduction	40
5.2	Backend - Static Analysis Pipeline	40
5.3	Web Frontend	42
5.3.1	Gemini assistant in the pseudo-C panel	44
5.4	WPF Desktop Application	44
5.5	VM Agent and Hyper-V Sandbox	46
5.5.1	Path A - Backend + HTTP vm-agent	46
5.5.2	Path B - PowerShell Scripts	47
5.6	User Manual	50
5.6.1	Web Interface	50
5.6.2	WPF desktop application	50
5.6.3	Hyper-V Sandbox	50
5.6.4	Quick CLI	51
5.7	Experimental Evaluation	51
5.7.1	Methodology and ethical considerations	51
5.7.2	Static scoring validation	51
5.7.3	Proxy detection metrics	52
5.7.4	Benign corpus static evaluation	52
5.7.5	Public IOC metadata reference	53
5.7.6	Optional MalwareBazaar static batch (implemented, not executed)	54
5.7.7	Dynamic sandbox validation	55
5.7.8	Formal performance benchmark	55
5.8	Tests	55
5.8.1	Repository documentation	56
5.9	Conclusions	56
6	Conclusions and Future Work	57
6.1	Main Conclusions	57
6.2	Limitations	57
6.3	Future Work	58
A	Appendix	60
A.1	Token verification in vm-agent	60

CONTENTS

A.2	RunGate - serialised execution	60
A.3	Hyper-V driver - agent HTTP client	61
A.4	vm-agent limits	61
A.5	Report transfer protocol (Path B)	62
A.5.1	Current implementation	62
A.5.2	Previous approaches (archive)	62
	Bibliography	63

List of Figures

4.1	Use case diagram - RAT ANALYZER	27
4.2	Activity diagram - static analysis pipeline	28
4.3	Activity diagram - dynamic analysis pipeline	29
4.4	Conceptual architecture of RAT ANALYZER	30
4.5	Integrated sequence diagram - static, dynamic, and WPF flows	31
4.6	Trust zones - host / sandbox / guest isolation	32
4.7	Authentication map - tokens and credentials	33
4.8	Comparison of paths A (HTTP vm-agent) and B (Hyper-V PowerShell)	33
4.9	Sequence - Path A (HTTP vm-agent)	34
4.10	Sequence - Path B (PsDirect + SHA256 verification)	35
5.1	Web interface - upload zone (<i>Drop & Analyze</i>)	43
5.2	Web interface - report, pseudo-C/IL, and risk score	44
5.3	WPF desktop application - main screen	46
5.4	Hyper-V - VM MalwareSandbox and snapshot CleanState	50

List of Tables

2.1	Comparison of the RAT ANALYZER solution with existing alternatives	16
3.1	Technology choices and rejected alternatives	19
4.1	Functional requirements of RAT ANALYZER	21
4.2	Specification table for FR 01	22
4.3	Specification table for FR 02	23
4.4	Specification table for FR 03	23
4.5	Specification table for FR 04	23
4.6	Specification table for FR 05	24
4.7	Specification table for FR 06	24
4.8	Specification table for FR 07	24
4.9	Specification table for FR 08	24
4.10	Specification table for FR 09	25
4.11	Traceability matrix - functional requirements, components, and tests . .	26
4.12	Security principles and implementation	36
4.13	Authentication by component	36
4.14	Attack surfaces and structural mitigations	37
4.15	Technical comparison - Path A (vm-agent) vs Path B (PowerShell) . . .	38
4.16	Job lifecycle (<code>analysis_jobs.py</code>)	38
4.17	Risk score factors (0–100)	39
5.1	Use case 1 - Suspicious sample analysis process	40
5.2	Main backend REST endpoints	41
5.3	HTTP upload controls (<code>upload_security.py</code>)	41
5.4	Environment variables - dynamic sandbox (Path A)	48
5.5	Fields of <code>guest_analysis_done.txt</code> (PsDirect protocol)	49
5.6	Static scoring validation scenarios	51
5.7	Proxy detection metrics on labeled synthetic profiles ($n = 59$)	52
5.8	Severity breakdown of synthetic profiles ($n = 59$; predicted matches expected)	52
5.9	Representative synthetic profiles (subset of $n = 59$)	53
5.10	Benign corpus static evaluation ($n = 8$ PEs; measured on development host)	53
5.11	Public IOC metadata reference (MalwareBazaar listings; no host-side PE analysis)	54
5.12	Formal benchmark - static pipeline and dynamic sandbox paths	55
5.13	Summary of automated tests	56
A.1	Execution controls in vm-agent (<code>AgentLimits.cs</code> , <code>RunGate.cs</code>)	61

List of code excerpts

3.1	VM driver selection - <code>vm_orchestrator.py</code>	18
4.1	Hyper-V name allowlist - <code>vm_drivers/hyperv.py</code>	35
4.2	Safe sample path resolution - <code>vm-agent/Services/SampleStorage.cs</code>	36
5.1	Job creation endpoint - <code>routers/jobs.py</code>	42
5.2	Publishing VM report to backend - <code>DynamicAnalysisService.cs</code> . . .	45
5.3	Hyper-V driver orchestration - <code>vm_drivers/hyperv.py</code>	46
5.4	Serialised execution endpoint - <code>AgentEndpoints.cs</code>	47
5.5	Sample execution with timeout - <code>SampleRunner.cs</code>	47
5.6	Guest completion marker - <code>vm/Run-MalwareAnalysis.ps1</code>	48
5.7	Post-transfer SHA-256 verification - <code>SandboxCommon/VmFileTransfer.ps1</code>	48
5.8	Static analysis via CLI	51
A.1	Authentication middleware - <code>vm-agent/Security/AgentTokenMiddleware.cs</code>	60
A.2	Single-run gate - <code>vm-agent/State/RunGate.cs</code>	60
A.3	VM agent HTTP calls - <code>vm_drivers/hyperv.py</code>	61

Acronyms

API	Application Programming Interface
C2	Command and Control (command-and-control server)
CLI	Command Line Interface
DLL	Dynamic Link Library
ETW	Event Tracing for Windows
FR	Functional Requirement
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
IL	Intermediate Language (.NET)
JSON	JavaScript Object Notation
LLM	Large Language Model
NFR	Non-Functional Requirement
NDJSON	Newline-Delimited JSON (event streaming)
PE	Portable Executable
PsDirect	PowerShell Direct (remote access to Hyper-V guest)
RAT	Remote Access Trojan
REST	Representational State Transfer
RQ	Redis Queue
SQL	Structured Query Language
UBI	Universidade da Beira Interior
UUID	Universally Unique Identifier
VM	Virtual Machine
SHA-256	Secure Hash Algorithm 256 bits (file integrity)
WPF	Windows Presentation Foundation
YARA	Yet Another Recursive Acronym (detection rule language)

Chapter

Introduction

1.1 Context

This report presents the final-year project in Computer Science Engineering at Universidade da Beira Interior, dedicated to the development of RAT ANALYZER: an integrated static and dynamic analysis platform for detecting *Remote Access Trojans* (RATs) in Windows executables (.exe, .dll) and, with reduced scope, C# source files (.cs): on the web/API, .cs files undergo string-based heuristics only (no PE imports or ILSpy); compilation to .exe is available via the optional Tkinter GUI or manual build before upload.

RATs are among the most concerning classes of malware today: they allow an attacker to remotely control a compromised system, harvest credentials, exfiltrate data, and execute arbitrary commands. Their detection often requires combining *offline* techniques (static binary analysis) and *online* techniques (behaviour observation in a controlled environment), complemented by signature rules and scoring heuristics.

RAT ANALYZER was designed in an academic context as an **educational and triage** tool: in addition to flagging risk indicators, it exposes decompiled code, structured reports, and explanations of suspicious functions, enabling students and junior analysts to understand *why* a file is considered malicious. Complementarily, the web interface integrates a **Google Gemini**-based assistant that allows users to ask, in natural language, about the meaning of the pseudo-C excerpt visible in the results panel.

1.2 Motivation

The proliferation of malware with remote access capabilities demands accessible tools that integrate multiple analysis layers into a single workflow. Commercial solutions such as cloud sandboxes (VirusTotal, Any.Run, Hybrid Analysis) are powerful but often unsuitable for teaching or local laboratory contexts, whether due to cost or sample confidentiality.

The motivation for this project rests on four pillars:

- **Integration** - Unify static analysis, YARA rules, decompilation, and sandbox execution in a single workflow (API, web interface, desktop application).
- **Isolation** - Run potentially malicious samples in a Hyper-V VM without Internet connectivity, with automatic restore to a clean snapshot.

- **Transparency** - Produce detailed reports (imports, C2 strings, evasion, risk score, pseudo-C/IL) instead of a simple binary classification; optionally, support pseudo-C interpretation with an LLM assistant in the frontend.
- **Academic reproducibility** - Open source, versioned documentation, automated tests (pytest, Vitest, GitHub Actions CI), and security documentation (docs/SEGURANCA.md).

1.3 Objectives

The main objectives of the project are:

1. Develop a Python **backend** (FastAPI) capable of receiving samples, running a complete static analysis pipeline, and orchestrating dynamic analysis jobs (**static**, **dynamic**, or **both**).
2. Implement a **web interface** (React + Vite + TypeScript) with a *drag-and-drop* zone, analysis modes (**static**, **dynamic**, **both**), progress streaming, report visualisation, pseudo-C (Ghidra) and IL (.NET), a Gemini assistant to explain C code excerpts, and bilingual support (Portuguese/English).
3. Provide a **WPF desktop application** (.NET 8) as an alternative entry point, including dependency bootstrap, persistent language selection, and integration with the Hyper-V sandbox (Path B).
4. Build a **VM agent** (Minimal API .NET) and PowerShell scripts for isolated sample execution and behavioural telemetry collection.
5. Define a **scoring model** (0–100) that aggregates multiple indicators (YARA, C2, evasion, packers, obfuscation, persistence).
6. Validate the system with automated tests, benign samples (**benign-vm-test**), and operational documentation for local deployment.

1.4 Document Organisation

To demonstrate the stages of the work carried out, this report is organised as follows:

1. The first chapter — *Introduction* — presents the project, motivation, context, objectives, and document structure.
2. The second chapter — *State of the Art* — analyses malware analysis approaches and compares existing solutions with the developed platform.
3. The third chapter — *Technologies and Tools Used* — describes the technology stack adopted throughout the development cycle.

CHAPTER 1. INTRODUCTION

4. The fourth chapter — *Software Engineering* — presents requirements, diagrams (use cases, activity, architecture, sequence, trust zones, authentication, and sandbox paths), data model, and security architecture.
5. The fifth chapter — *Implementation* — details the implemented components, experimental evaluation, user manual, and tests performed.
6. The sixth chapter — *Conclusions and Future Work* — summarises the results obtained and proposes future developments.
7. The *Appendix* includes relevant code examples.
8. The *Bibliography* collects the references cited throughout the document.

Chapter

State of the Art

2.1 Background

This chapter frames the application of malware analysis techniques in the cybersecurity domain, with specific focus on RAT detection. It also examines the most widely used existing solutions in this context — static, dynamic, and hybrid — comparing them with the approach adopted in RAT ANALYZER.

2.2 Malware Analysis: Overview

Malware analysis is traditionally divided into two complementary approaches [1, 2, 16, 17]:

- **Static analysis** - Examines the file without executing it: PE structure, sections, imports/exports, strings, entropy, YARA signatures, decompilation, and detection of evasion techniques (anti-debug, anti-VM).
- **Dynamic analysis** - Runs the sample in an isolated sandbox and monitors behaviour: process creation, registry changes, files, network, and persistence.

In the context of RATs, combining both approaches is especially relevant: static analysis allows identification of suspicious imports (webcam access, keylogging, exfiltration via `ws2_32.dll`) and C2 strings without execution risk, while dynamic analysis reveals actual behaviour — established network connections, files created, persistence registry keys.

2.3 Trends in RAT and Malware Detection

This section explores the most widely used and successful approaches for malware detection and analysis, with particular attention to those closest to the RAT domain.

2.3.1 Commercial and cloud sandboxes

Platforms such as *Any.Run* [3], *Hybrid Analysis* [4], and *Joe Sandbox* [5] offer complete dynamic analysis with interactive reports, building on earlier automated sandbox research [18, 19]. These platforms are powerful and rich in telemetry but present

relevant limitations for academic and laboratory contexts: high cost on professional plans, the need to submit samples to external servers (compromising the confidentiality of sensitive samples), and inability to customise the execution environment. RAT ANALYZER was designed precisely to address these gaps: local execution, complete isolation, and zero cost.

2.3.2 VirusTotal

VirusTotal [6] aggregates more than 70 antivirus engines and provides static metadata and behavioural heuristics. It is the most widely used triage tool worldwide. However, sample submission shares them with the platform’s partners, which is unacceptable in forensic investigation or proprietary sample analysis contexts. Furthermore, it does not expose decompiled code or allow customisation of detection rules.

2.3.3 Open-source frameworks

- **Cuckoo Sandbox** [7] — Self-hostable, extensible Python sandbox with support for behavioural analysis on Windows and Linux. It is the open-source reference closest to RAT ANALYZER, but requires complex configuration and does not integrate decompilation or explainable scoring.
- **CAPE Sandbox** [8] — Cuckoo fork focused on extracting configurations from malware families and C2 servers. Highly specialised; not suitable as a generalist educational tool.
- **YARA** [9] — Rule language for pattern-based detection, used by virtually all analysis platforms. RAT ANALYZER integrates YARA as one phase of the static pipeline, combining it with scoring and decompilation.
- **Ghidra / ILSpy** [10, 11] — Decompilers for native binaries and .NET, respectively. Essential tools in manual analysis, but typically used in isolation on the command line. RAT ANALYZER orchestrates both and exposes the results directly in the web interface.

2.3.4 Machine learning-based detection

Recent work applies *deep learning* to static PE file features for automatic malware classification [12, 13, 20, 21]. These approaches achieve high classification metrics but produce opaque decisions — the analyst knows the file was classified as malicious, but not why. RAT ANALYZER prioritises **explainability** through concrete indicators (suspicious imports, C2 strings, YARA results, pseudo-C) and an optional LLM assistant that helps interpret decompiled code. Public threat-intelligence repositories such as MalwareBazaar [22] complement local analysis by providing family labels and SHA-256 hashes without requiring sample execution in the academic repository. Large-scale malware labeling tools such as AVclass [23] illustrate how family attribution at scale complements local triage, while ransomware and trojan surveys [24] contextualise the broader remote-access threat landscape that RAT families inhabit.

2.4 Positioning relative to existing solutions

For this project, a distinct approach was chosen from commercial solutions and existing generic frameworks. Instead of replicating a conventional sandbox, RAT ANALYZER combines multi-phase static analysis with a native Hyper-V sandbox and integrated decompilation, exposed through a web interface and a WPF desktop application.

Table 2.1: Comparison of the RAT ANALYZER solution with existing alternatives

Reference	Technology	RAT Analyzer solution
VirusTotal	Cloud multi-AV	Local; YARA + heuristics + Hyper-V sandbox
Cuckoo/CAPE	Self-hostable sandbox	Native Windows sandbox (Hyper-V)
Any.Run / Hybrid	Commercial sandbox	Free; educational focus; pseudo-C/IL
YARA standalone	Signature rules	YARA + scoring + decompilation
Ghidra/ILSpy CLI	Offline decompilation	Orchestrated decompilation + web UI
ML classifiers	Opaque models	Explainable indicators + report + optional LLM assistant

In summary, Hyper-V avoids external sandbox infrastructure on Windows laboratory hosts; Ghidra and ILSpy are invoked from the pipeline instead of separate manual steps; and the optional Gemini assistant can explain pseudo-C excerpts chosen by the user in the browser.

2.5 Conclusions

The state of the art confirms that effective RAT detection benefits from multi-layer analysis. Commercial solutions are powerful but inaccessible in academic contexts; open-source frameworks are flexible but require significant manual integration. RAT ANALYZER targets local use in university laboratories and cybersecurity training, with explainable indicators rather than opaque classification alone.

Chapter

Technologies and Tools Used

3.1 Introduction

This chapter summarises the main technologies and tools used in the development of RAT ANALYZER, namely in building the backend, user interfaces, dynamic analysis sandbox, and project report.

3.2 Python and FastAPI

Python 3.10+ was adopted as the main backend language, due to its maturity in the information security ecosystem and the wide availability of libraries for PE file analysis, binary format parsing, and integration with decompilation tools. **FastAPI** [15] was chosen as the web framework for providing automatic data validation with Pydantic, interactive OpenAPI documentation (`/docs`), and native support for asynchronous NDJSON streaming (`/api/analyze_stream`), essential for real-time feedback during analysis. There is also an optional graphical interface in **Tkinter** (`backend/gui/rat_analyzer_gui.py`) for local static analysis without a browser, useful for quick demonstrations.

3.3 React, Vite and TypeScript

The web interface was built with **React 18** and **TypeScript**, ensuring static typing that reduces errors during development. The **Vite** bundler enables instant hot-reload during development and optimised production builds. For visual styling, **Tailwind CSS** and **shadcn/ui** were adopted; transition animations are handled with **Framer Motion**. The unit test suite uses **Vitest** (62 unit tests). Integration with **Google Gemini** (`gemini-2.0-flash`) is done directly in the browser: the user's API key is stored in `localStorage` and never passes through the FastAPI backend, ensuring the server has no access to third-party credentials.

3.4 .NET 8, WPF and Minimal API

Three .NET 8 projects share the `RatAnalyzer.sln` solution: **Rat Analyzer** (WPF desktop, compiles to `Rat Analyzer.exe`), **VmAgent** (Minimal API running inside the sandbox VM to receive and execute samples), and **BenignVmTest** (benign executable to validate the dynamic pipeline without risk). WPF serves as an alternative entry

point to the web interface, with automatic dependency bootstrap, persistent language selection, and direct integration with the Hyper-V sandbox (Path B).

3.5 YARA, Ghidra and ILSpy

YARA [9] is the pattern-matching rule language used for signature-based detection. Rules reside in `yara_rules/` and are compiled and applied to the binary under analysis. **ILSpy CLI** [11] decompiles .NET assemblies into readable C# code, enabling inspection of executables written with the managed runtime. **Ghidra 12+** [10], developed by the NSA, is used for decompilation of native PE binaries, producing pseudo-C that is exposed in the web interface code panel.

3.6 Hyper-V, PowerShell and SQLite

The dynamic sandbox relies on **Hyper-V**, the native Windows hypervisor, with a VM named **MalwareSandbox**, connected to an **Internal** switch with no Internet connectivity on the guest and a **CleanState** snapshot that is restored before each analysis. **PowerShell** automates one-time VM setup (`01-Setup-MalwareSandbox.ps1`) and, on **Path B**, sample copy, guest execution, and report transfer (`04-Run-Sample.ps1`). On **Path A**, the Python backend (`vm_drivers/hyperv.py`) uses PowerShell only for snapshot restore and VM start; sample upload, execution, and report retrieval go through the HTTP vm-agent. **SQLite** persists analysis jobs in `analysis.db`; **Redis** with **RQ** is a configurable option for distributed queuing in environments with multiple workers.

The backend selects the sandbox driver at runtime via `SANDBOX_VM_DRIVER`; the orchestrator delegates to `HyperVVMDriver` without changing the rest of the job pipeline:

```

1 def run_dynamic_analysis(job: AnalysisJob) -> Dict[str, Any]:
2     driver = _build_driver() # stub / hyperv / proxmox (
        experimental)
3     out: DynamicAnalysisOutput = driver.run(job)
4     return {"summary": out.summary, "behavior": out.behavior, "
        driver": driver.name}
```

Code excerpt 3.1: VM driver selection - `vm_orchestrator.py`

3.7 GitHub and LaTeX

GitHub hosts the project repository, with Git version control and continuous integration via **GitHub Actions** (CI with `pytest`, `Vitest`, and `dotnet build`). Changes are recorded in `CHANGELOG.md` and development conventions in the root `README.md`. This report was written in **LaTeX**, a typesetting system widely used in academic and scientific contexts, which automatically manages section numbering, figures, tables, and bibliographic references with rigour and consistency.

3.8 Security and Isolation

The design adopts *fail-closed* in production: the backend refuses to start without `RATANALYZER_API_TOKEN` when `RATANALYZER_ENV=production`; the vm-agent refuses to start without `VM_AGENT_TOKEN`, except when `VM_AGENT_ALLOW_INSECURE=1` in local development.

Secrets reside in `secrets/` (gitignored) and are produced by `generate-production-secrets.ps1` under `scripts/`. HTTP uploads require authentication, with name sanitisation, size limits, and storage in `DATA_DIR` outside the repository. The canonical security documentation is in `docs/SEGURANCA.md`.

3.9 Technology choices and rejected alternatives

Table 3.1 summarises major stack decisions and why simpler or more common alternatives were not adopted. The goal was a **local, Windows-native** platform for academic laboratories rather than a cloud-first or containerised product.

Table 3.1: Technology choices and rejected alternatives

Area	Chosen	Rejected	Rationale
Sandbox	Hyper-V + snapshot	Docker, Cuckoo VM	Native Windows telemetry; no Linux guest for PE/RAT behaviour
API	FastAPI + Pydantic	Flask, Django	Async NDJSON streaming; automatic OpenAPI
Frontend	React + Vite + TS	Vue, Angular	Team familiarity; Vitest/Playwright ecosystem
Desktop	WPF (.NET 8)	Electron, WinForms	Direct Hyper-V/PsDirect; lighter than Electron
Queue	Optional Redis/RQ	Mandatory Celery	Works offline with local threads (NFR 06)
Decompiler	ILSpy + Ghidra	IDA Pro only	Open source; IDA deferred to future work
LLM assistant	Gemini client-side	Backend proxy	Sample confidentiality; key stays in browser

Docker was explicitly excluded: malware sandboxes require GUI/process behaviour on Windows, and Hyper-V isolation is already available on laboratory hosts. A backend LLM proxy would centralise API keys but would route pseudo-C excerpts through the server; the client-side design keeps decompiled code local except for the excerpt the user chooses to send.

3.10 Conclusion

The multi-language stack reflects the heterogeneous nature of malware analysis: Python for pipeline and API, TypeScript for user experience, C# for integration with the Windows and Hyper-V ecosystem, and PowerShell for infrastructure automation. The combination of these technologies support a separation of responsibilities between pipeline, UI, and sandbox automation; documentation is versioned in the repository.

Chapter

Software Engineering

4.1 Introduction

This chapter aims to detail the decisions taken regarding system architecture and design during the planning phase. The first section presents requirements analysis, divided into functional and non-functional requirements, followed by use case, activity, architecture, sequence, trust zone, and sandbox path diagrams. The data model and adopted security architecture are also presented, explaining how the different components interact and protect each other.

4.2 Requirements Analysis

4.2.1 Functional Requirements

The following Functional Requirements (FR) were defined taking into account user and system needs:

Table 4.1: Functional requirements of RAT ANALYZER

ID	Description
FR 01	Sample upload and validation. The system must accept <code>.exe</code> , <code>.dll</code> , and <code>.cs</code> files via web interface, WPF desktop, or REST API. For <code>.cs</code> , the web/API path applies string heuristics only; full PE/ILSpy analysis requires a compiled <code>.exe/.dll</code> . Validates extension, maximum size (<code>RATANALYZER_MAX_UPLOAD_MB</code> , default 100 MB), and sanitises file names.
FR 02	Complete static analysis. 7-phase pipeline: PE metadata, ILSpy (.NET, if applicable), static analysis (imports, C2 strings, evasion), deobfuscation, YARA scan, scoring, native disassembly/Ghidra (when ILSpy is unavailable), and report.
FR 03	Dynamic analysis in sandbox. Run sample in an isolated Hyper-V VM (no Internet), monitor behaviour, and return behavioural report. Supports <code>static</code> , <code>dynamic</code> , and <code>both</code> modes (web: Path A via <code>POST /api/analysis</code> ; WPF: Path B).
FR 04	Job management. Create, query, and list jobs (<code>POST/GET /api/analysis</code>). Persistence in SQLite with UUID v4; artefacts in <code>sandbox_jobs/</code> .

ID	Description
FR 05	Results visualisation. Web interface with three panels: pseudo-C (Ghidra), IL (.NET), and <i>report</i> (split horizontally into static + VM when both exist in the same <code>jobId</code>); contextual navigation between indicators and code, including the Xref Explorer (<code>/analysis/{jobId}/xref</code>). Permalink <code>/analysis/{jobId}</code> with real-time updates while VM analysis is in progress.
FR 06	Risk scoring. Calculate score 0–100 aggregating YARA, C2, evasion, packers, obfuscation, persistence, and entropy; classify as CRÍTICO/ALTO/MÉDIO/BAIXO/MUITO BAIXO with strong_evidence gates for the two highest levels.
FR 07	Progress streaming. <code>/api/analyze_stream</code> endpoint in NDJSON with incremental logs during static analysis.
FR 08	VM orchestration. Two dynamic paths: (A) Python backend + HTTP vm-agent; (B) PowerShell scripts + PsDirect/Copy-VMFile copy with SHA256 verification (WPF). On Path B, the report transferred to the host is published via <code>POST /api/analysis/upload_dynamic</code> , reusing the static analysis <code>jobId</code> when applicable.
FR 09	Gemini assistant in pseudo-C. In the decompiled C code panel, the user can configure a Google AI Studio API key (local storage) and ask, via chat, what the visible excerpt does. Submission respects text limits (code and question); a demonstration mode with simulated response exists, without API call.

Table 4.2 details the specification for requirement FR 01.

Table 4.2: Specification table for FR 01

FR 01	Sample upload and validation
Function	Receive and validate <code>.exe</code> , <code>.dll</code> , or <code>.cs</code> files submitted by the user.
Inputs	Binary file or source code; analysis mode (static , dynamic , both). For <code>.cs</code> on web/API, only static string heuristics apply; dynamic analysis expects an executable.
Outputs	Created job UUID; descriptive error message if rejected.
Action	Validate extension and size; sanitise name; create database record; return <code>job_id</code> .
Preconditions	Backend running; valid authentication token.
Side effects	File stored in <code>DATA_DIR/sandbox_jobs/{job_id}/</code> .

Tables 4.5 to 4.10 summarise the remaining functional requirements.

Table 4.3: Specification table for FR 02

FR 02	Complete static analysis
Function	Execute 7-phase static analysis pipeline on the submitted sample.
Inputs	Job ID; file stored in <code>DATA_DIR</code> .
Outputs	JSON report with PE metadata, suspicious imports, C2 strings, risk score, YARA results, and decompiled code excerpts.
Action	Sequential phases: PE metadata → ILSpy (if <code>.exe/.dll</code>) → static analysis → deobfuscation → YARA → scoring → native disassembly/Ghidra (if needed) → report.
Preconditions	FR 01 completed successfully; ILSpy and Ghidra installed (for decompilation).
Side effects	Report saved in <code>last_analysis.json</code> ; job updated to <code>completed</code> state.

Table 4.4: Specification table for FR 03

FR 03	Dynamic analysis in sandbox
Function	Run the sample in an isolated Hyper-V VM and collect behavioural telemetry.
Inputs	Job ID; sample path; configurable execution timeout.
Outputs	Behavioural report with created processes, registry changes, files, and network activity.
Action	Restore <code>CleanState</code> snapshot; start VM; transfer sample; execute; collect report; verify SHA-256 integrity.
Preconditions	Hyper-V active; <code>MalwareSandbox</code> VM exists; Internal switch without Internet.
Side effects	VM restored to clean snapshot after each run.

Table 4.5: Specification table for FR 04

FR 04	Job management
Function	Create, query, and list analysis jobs with a unique identifier.
Inputs	Validated file; type (<code>static dynamic both</code>).
Outputs	<code>jobId</code> , <code>state</code> , and <code>metadata</code> via <code>GET /api/analysis/{id}</code> .
Action	Persist in SQLite; store artefacts in <code>sandbox_jobs/</code> .
Preconditions	FR 01 completed; valid API token.

Table 4.6: Specification table for FR 05

FR 05	Results visualisation
Function	Display pseudo-C, IL, and static/dynamic report in the web interface.
Inputs	<code>jobId</code> ; JSON artefacts and code excerpts.
Outputs	Synchronised panels; permalink <code>/analysis/{jobId}</code> .
Action	Polling or streaming; horizontal static+VM split when both exist.
Preconditions	FR 02 and/or FR 03 completed successfully.

Table 4.7: Specification table for FR 06

FR 06	Risk scoring
Function	Aggregate indicators into a 0–100 score and qualitative level.
Inputs	Static results, YARA, and deobfuscation.
Outputs	<code>riskScore</code> , <code>riskLevel</code> , breakdown by factor.
Action	Apply per-factor caps (<code>risk_scorer.py</code>); classify level using score thresholds and <code>strong_evidence</code> (YARA, C2, imports).
Preconditions	FR 02 completed (analysis and YARA phases).

Table 4.8: Specification table for FR 07

FR 07	Progress streaming
Function	Emit NDJSON events during static analysis.
Inputs	Submitted file; API token.
Outputs	Incremental log stream and current phase.
Action	<code>POST /api/analyze_stream</code> ; close with final result.
Preconditions	Backend active; static pipeline available.

Table 4.9: Specification table for FR 08

FR 08	VM orchestration
Function	Run dynamic analysis on Path A (HTTP, web/API) or B (PowerShell, WPF).
Inputs	Sample; timeout; guest credentials (Path B).
Outputs	Behavioural report; SHA-256 integrity (Path B).
Action	Restore snapshot; transfer sample; collect telemetry.
Preconditions	Hyper-V active; <code>MalwareSandbox</code> VM; FR 01 completed.

Table 4.10: Specification table for FR 09

FR 09	Gemini assistant in pseudo-C
Function	Explain, in natural language, the visible pseudo-C excerpt.
Inputs	Code excerpt (max. 16 000 characters); user question.
Outputs	Textual response in the <i>chatbox</i> ; demonstration mode without API.
Action	Client-side call to <code>gemini-2.0-flash</code> ; key in <code>localStorage</code> .
Preconditions	C code panel with content; modern browser.

4.2.2 Non-Functional Requirements

The Non-Functional Requirements (NFR) were organised into the following categories, taking into account the quality attributes most relevant to the malware analysis context:

NFR 01 - Security

The API must bind only to the local address (127.0.0.1); authentication tokens mandatory in production (`RATANALYZER_API_TOKEN`, `VM_AGENT_TOKEN`); secrets never versioned in the Git repository.

NFR 02 - Isolation

The sandbox VM must be connected to an **Internal** switch with no Internet routing to the guest; on **Path B**, a network *preflight* runs before each sample (`RunSample/PhaseA-Setup.ps1`); Path A relies on correct manual VM configuration; automatic restore to `CleanState` snapshot after each dynamic analysis.

NFR 03 - Usability

The web interface must support *drag-and-drop*; must provide real-time feedback during analysis; must be responsive and accessible in modern browsers. A user with basic information security knowledge should be able to submit and interpret an analysis without additional training.

NFR 04 - Performance

The system must support at least 2 concurrent workers in default mode (configurable); execution timeouts must be configurable per path: Path A defaults to 300 s (`SANDBOX_DYNAMIC_TIMEOUT_SECONDS`); Path B defaults to 120 s per sample (`TimeoutSeconds` in `04-Run-Sample.ps1`) with a 900 s global cap; VM restore/start operations use a separate 120 s timeout (`SANDBOX_VM_OP_TIMEOUT_SECONDS`).

NFR 05 - Maintainability

The project must include automated CI tests (pytest, Vitest, dotnet build); dependency locks with hashes for reproducible integrity; and technical documentation versioned in the repository.

NFR 06 - Availability

Redis queue mode must be optional and the system must work without it, falling back to local threads.

4.2.3 Requirements traceability

Table 4.11 maps each functional requirement to the main implementation modules and automated tests that verify it in CI. This matrix supports audit of coverage from specification to code.

Table 4.11: Traceability matrix - functional requirements, components, and tests

FR	Component(s)	Key modules / routes	Automated tests (CI)
FR 01	Backend API	upload_security.py, routers/jobs.py	test_upload_security.py, test_api.py (TestAnalyzeUpload)
FR 02	Static pipeline	rat_analyzer.py, modules/*	test_risk_scoring.py, test_yara_rules.py, test_pipeline_logging.py
FR 03	Sandbox	vm_orchestrator.py, Run-MalwareAnalysis.ps1	test_vm_drivers.py, benign-vm-test (xU- nit), Path B manual
FR 04	Job store	job_store.py, routers/jobs.py	test_job_reconstruction.py, test_api.py (TestListAnalyses)
FR 05	Web UI	Index.tsx, AnalysisResultsView.tsx	Vitest file-drop-zone.test.tsx, analysis.test.ts; Playwright E2E
FR 06	Scoring	risk_scorer.py	test_risk_scoring.py (TestRiskScorer)
FR 07	Streaming API	routers/analyze.py	test_api.py (token + upload contracts)
FR 08	VM orchestration	vm_drivers/hyperv.py, 04-Run-Sample.ps1, WPF	test_vm_drivers.py, VmAgent.Tests, test_upload_dynamic.py
FR 09	Gemini assistant	gemini.ts, GeminiAssistDialog.tsx	Vitest gemini.test.ts

4.3 Diagrams

The following diagrams were produced with PlantUML and automatically synchronised with the report via `sync_diagrams_to_report.py`. They cover functional flows, system architecture, security mechanisms, and the two sandbox paths, providing a complete view of the system at both the operational and isolation levels.

4.3.1 Use Case Diagram

To illustrate system operation, a use case diagram was created reflecting interactions between the analyst, web interface, WPF application, and backend. This diagram allows quick understanding of available actions and the actors involved in each flow. Figure 4.1 presents this diagram.

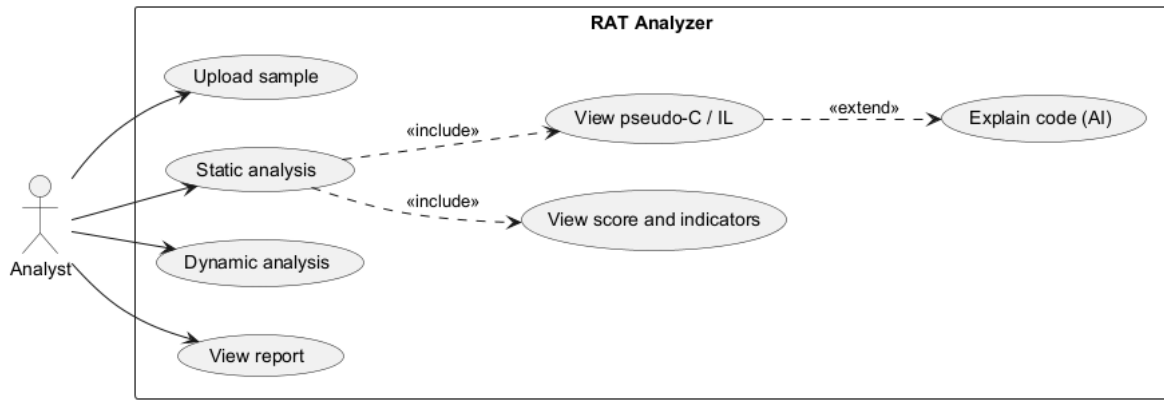


Figure 4.1: Use case diagram - RAT ANALYZER

4.3.2 Activity Diagram - Static Analysis

This section explains the different phases of the static analysis process and the actions executed at each stage. The pipeline processes the submitted file sequentially, with three decision points that may lead to early termination with error: file validation, decompiler availability, and successful YARA execution. Figure 4.2 summarises this pipeline, from upload to results presentation.

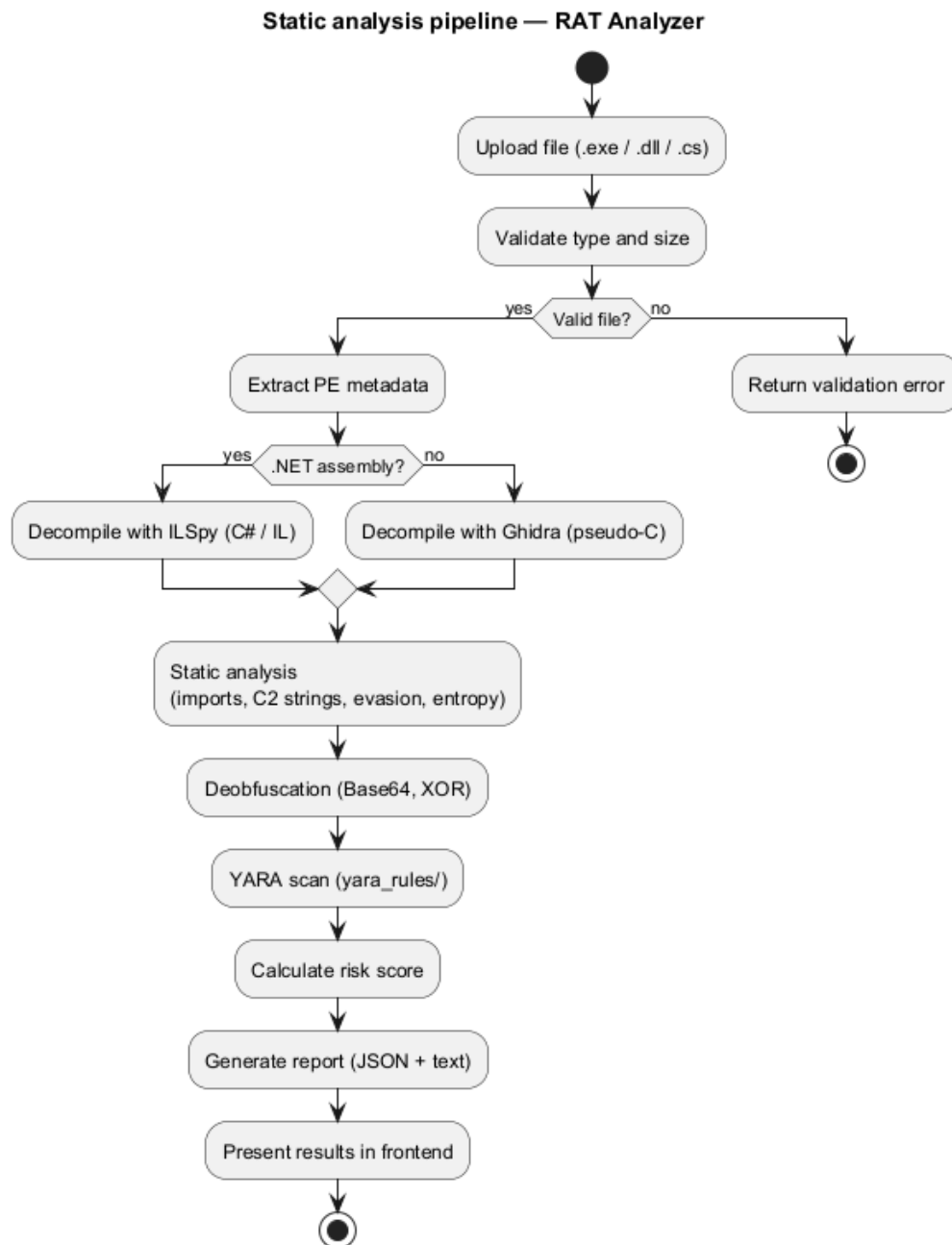


Figure 4.2: Activity diagram - static analysis pipeline

4.3.3 Activity Diagram - Dynamic Analysis

This section demonstrates how the sandbox execution process is orchestrated by the system. The flow covers two dynamic analysis paths: Path A (HTTP vm-agent, triggered from the web interface or API) and Path B (PowerShell Direct and SHA-256 verification via WPF). In either case, the system ensures restore of the clean snapshot before execution and validation of report integrity after transfer. Figure 4.3 describes this flow in detail.

Dynamic analysis pipeline — RAT Analyzer

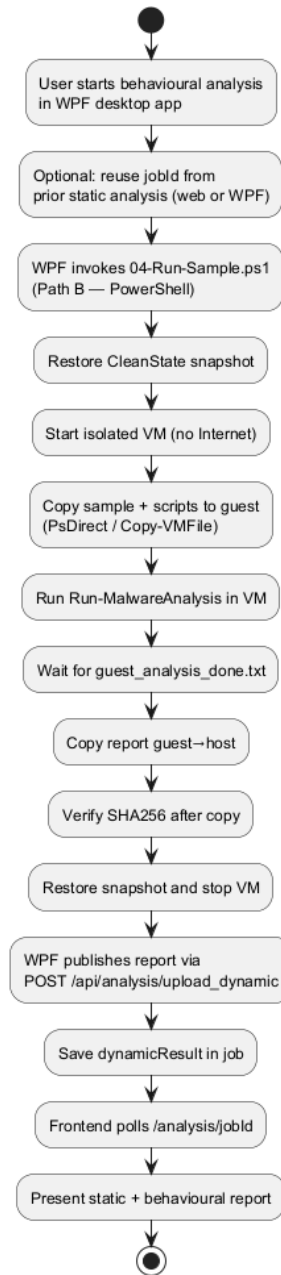


Figure 4.3: Activity diagram - dynamic analysis pipeline

4.3.4 System Architecture

Figure 4.4 presents the multi-layer architecture. Figure 4.5 details the integrated sequence between frontend, WPF, API, and sandbox.

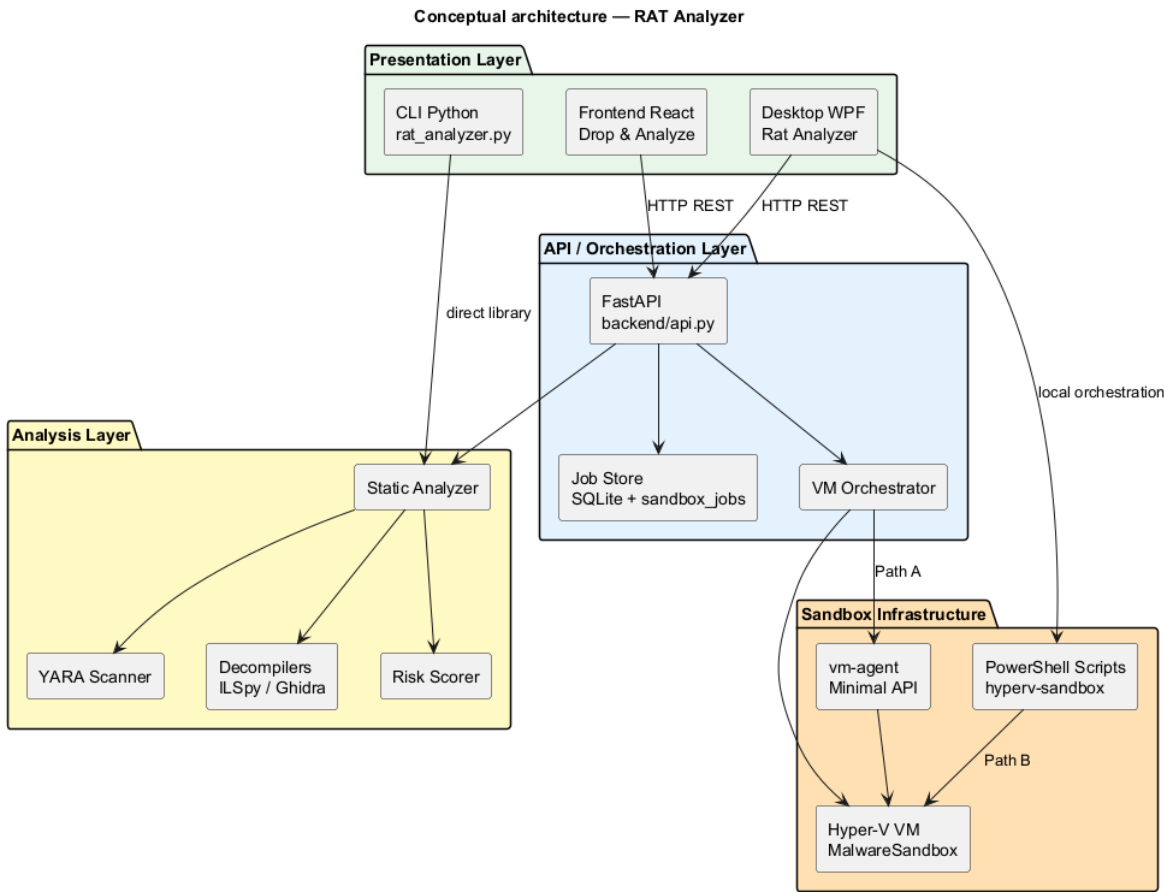


Figure 4.4: Conceptual architecture of RAT ANALYZER

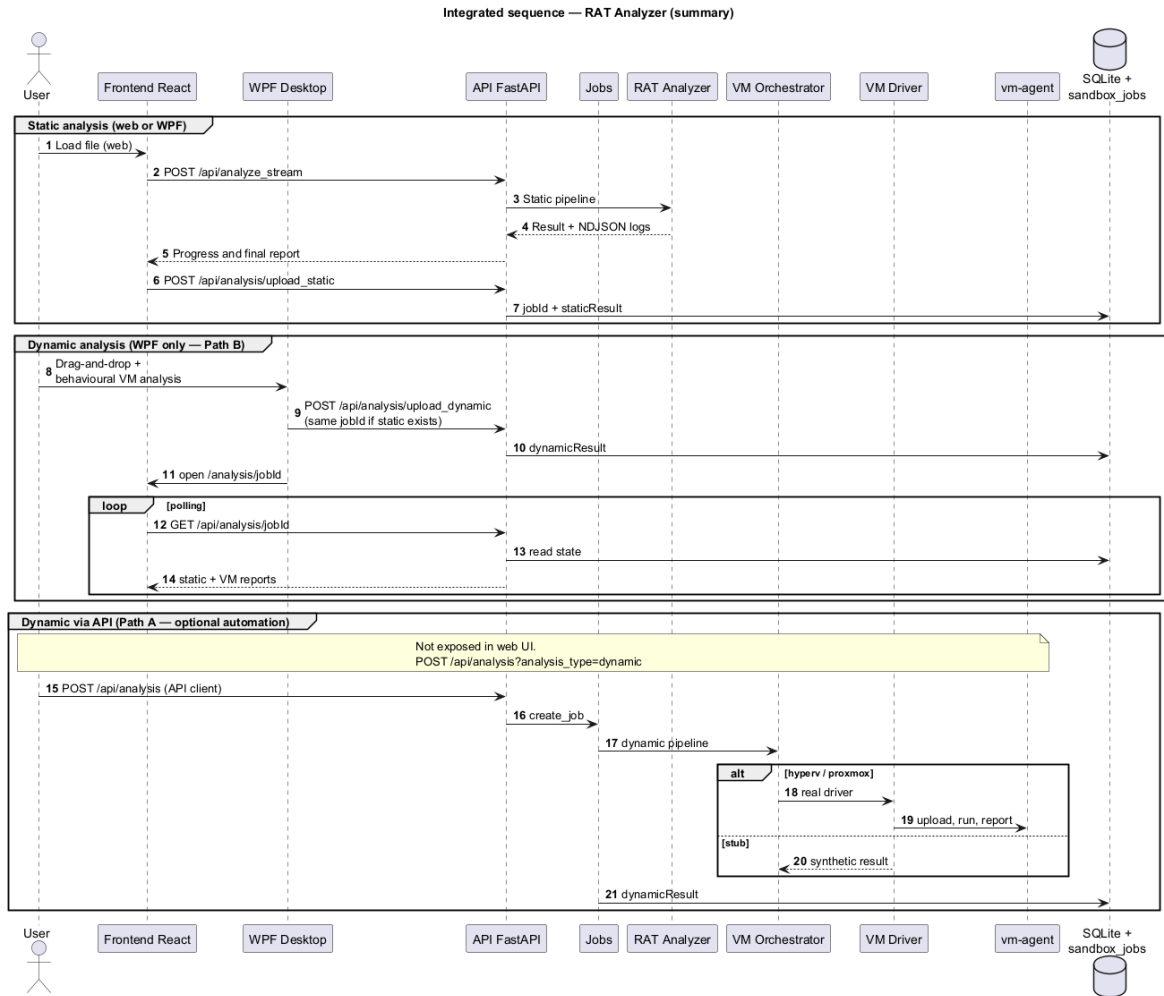


Figure 4.5: Integrated sequence diagram - static, dynamic, and WPF flows

4.3.5 Trust zones and authentication

Figure 4.6 summarises trust zones (host, sandbox network, guest). Figure 4.7 maps tokens and credentials between components.

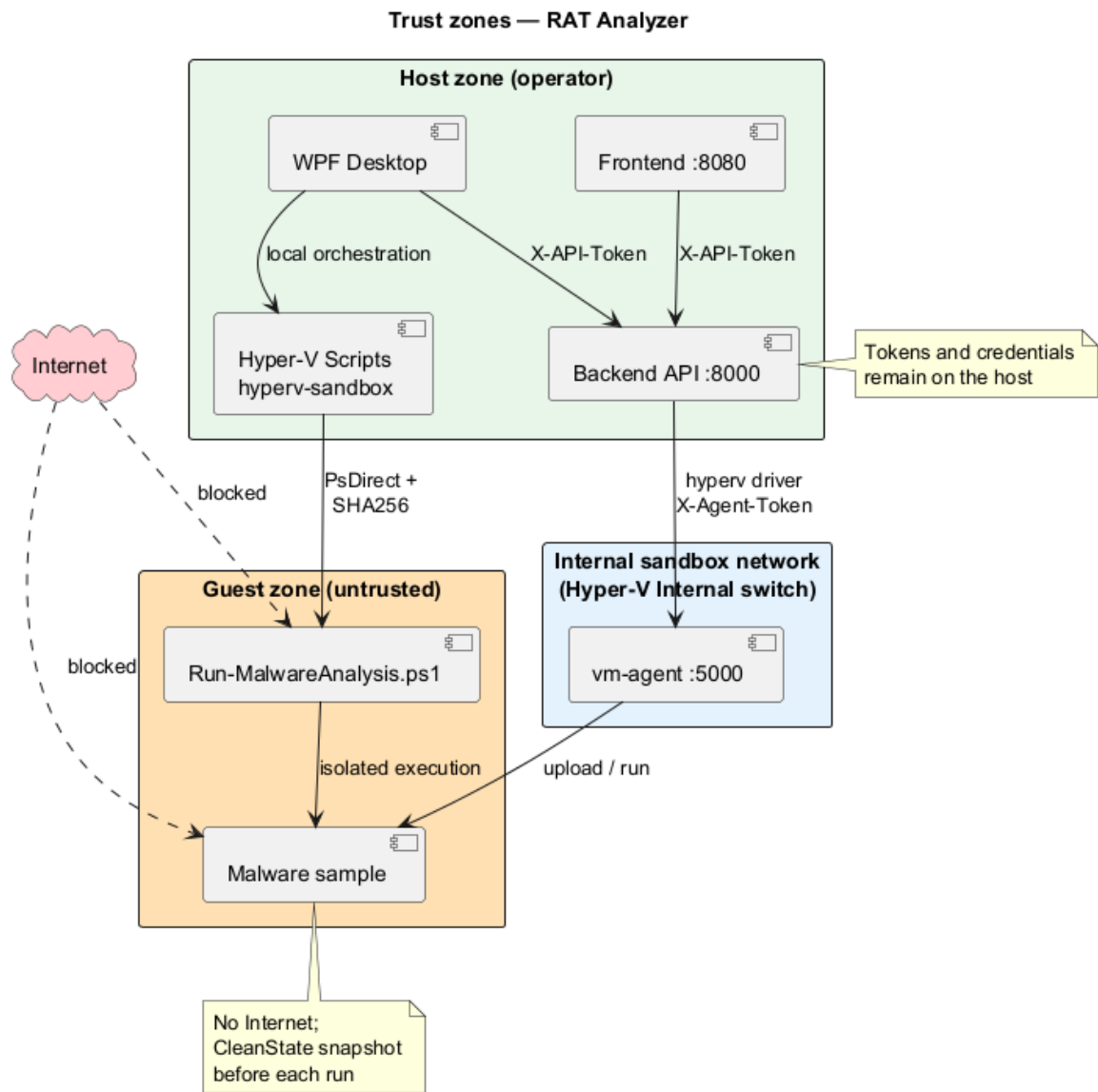


Figure 4.6: Trust zones - host / sandbox / guest isolation

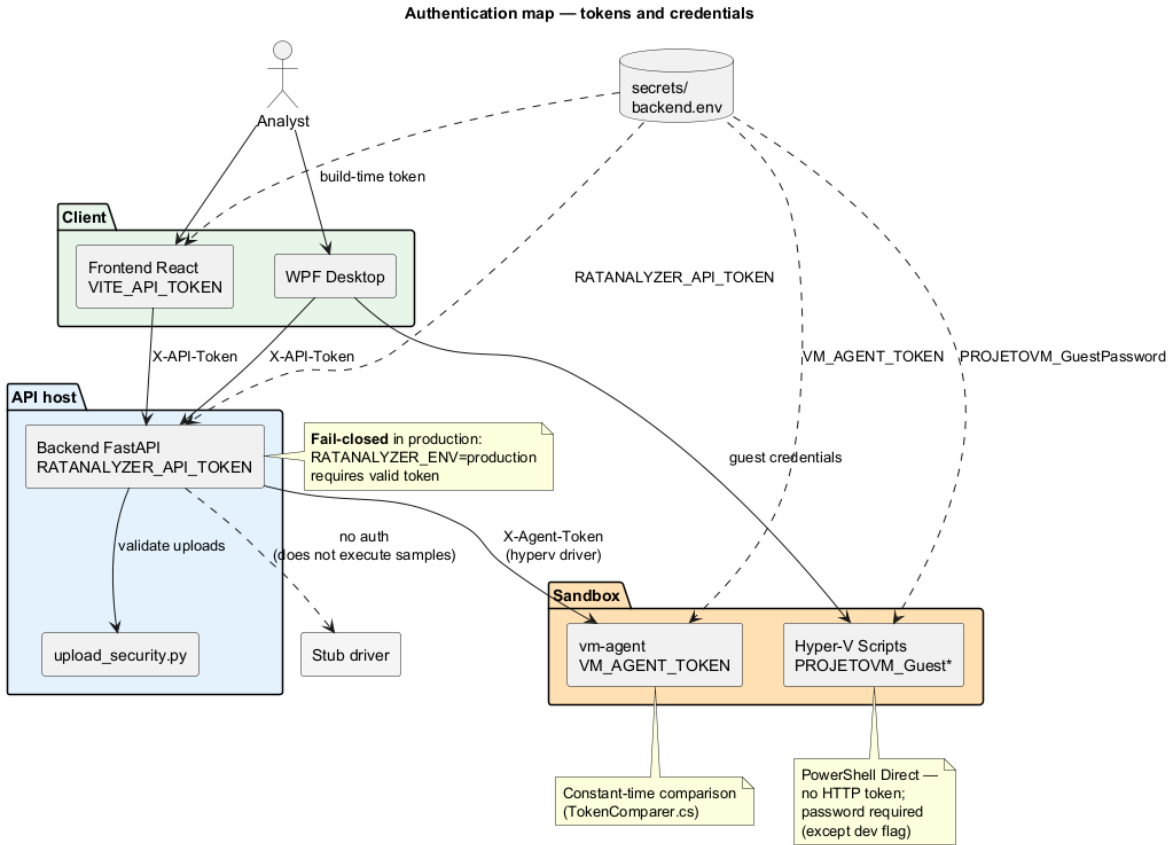


Figure 4.7: Authentication map - tokens and credentials

4.3.6 Sandbox paths (A and B)

Figure 4.8 compares the two independent dynamic analysis paths. Figures 4.9 and 4.10 detail each flow.

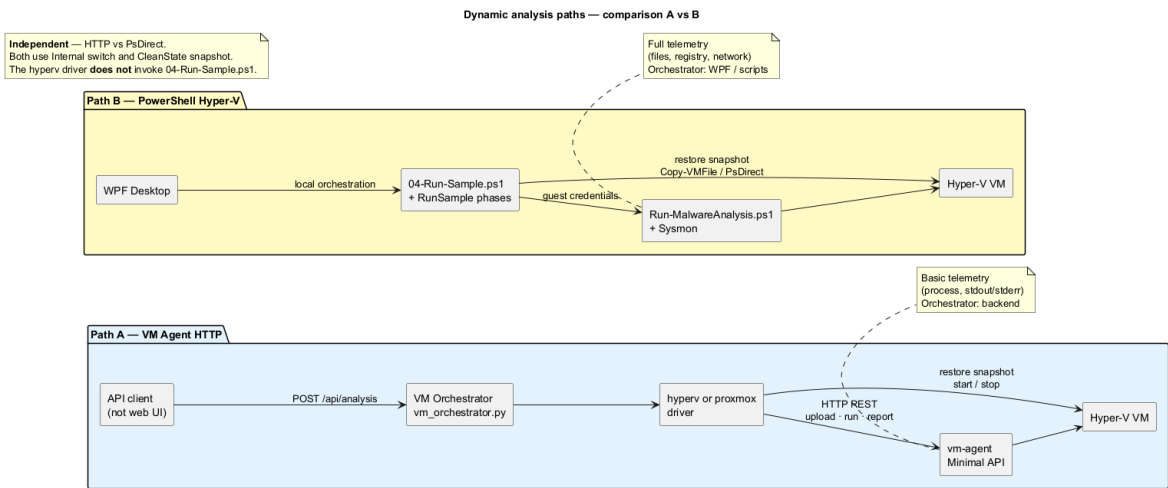


Figure 4.8: Comparison of paths A (HTTP vm-agent) and B (Hyper-V PowerShell)

CHAPTER 3. TECHNOLOGIES AND TOOLS USED

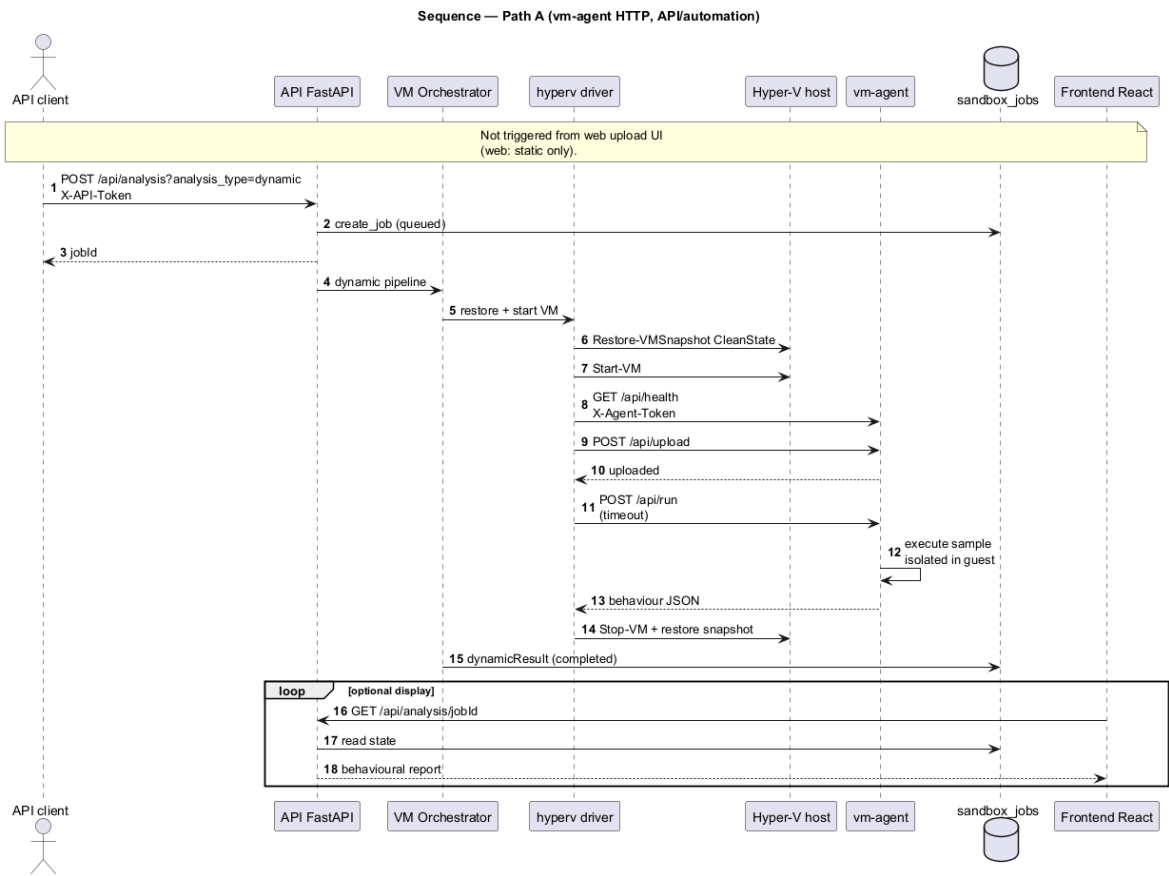


Figure 4.9: Sequence - Path A (HTTP vm-agent)

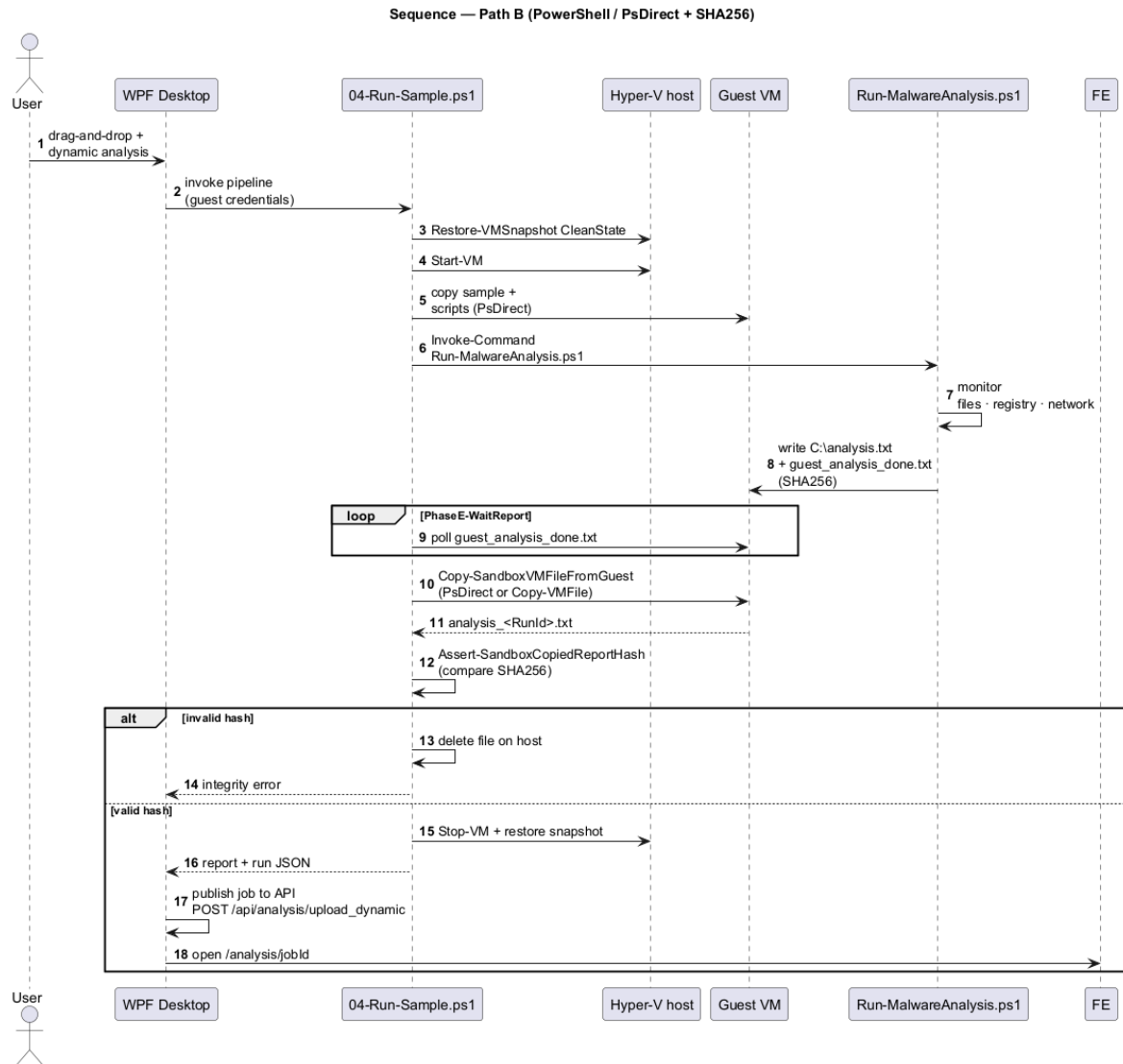


Figure 4.10: Sequence - Path B (PsDirect + SHA256 verification)

4.4 Security Architecture

Figure 4.6 models three zones: **host** (components under operator control), **sandbox network** (Hyper-V Internal switch, API↔vm-agent traffic), and **guest** (untrusted environment where the sample executes). Internet is not routed to the guest; each run begins with *restore* to the **CleanState** snapshot.

Figure 4.7 summarises authentication mechanisms. The backend validates **X-API-Token**; the vm-agent validates **X-Agent-Token** with constant-time comparison (**TokenComparer.cs**); Hyper-V scripts use guest credentials via PowerShell Direct. The **stub** driver does not execute samples and does not require sandbox authentication.

Before invoking **Restore-VMSnapshot** or **Start-VM**, the Hyper-V driver validates VM and snapshot names against a strict allowlist to block PowerShell injection (Listing 4.1):

```

1 _SAFE_PS_NAME_RE = re.compile(r"^[A-Za-z0-9 ._\-]+$")
2

```

```

3 def _validate_cfg(self) -> None:
4     if not sys.platform.startswith("win"):
5         raise RuntimeError("Driver Hyper-V s      suportado em
        Windows.")
6     for label, value in (
7         ("HYPERV_VM_NAME", self.cfg.vm_name),
8         ("HYPERV_SNAPSHOT_NAME", self.cfg.snapshot_name),
9     ):
10         if not _SAFE_PS_NAME_RE.match(value):
11             raise ValueError(f"{label} cont m caracteres
                inv lidos.")

```

Code excerpt 4.1: Hyper-V name allowlist - `vm_drivers/hyperv.py`

Table 4.12: Security principles and implementation

Principle	Implementation
Fail-closed	Backend refuses to start without token in produc- tion; vm-agent refuses without <code>VM_AGENT_TOKEN</code> (except <code>VM_AGENT_ALLOW_INSECURE=1</code> in dev)
Malware isolated	Execution only in Hyper-V VM or <code>stub</code> driver
Secrets outside Git	<code>secrets/</code> , <code>.env</code> gitignored
Local bind	API on 127.0.0.1; vm-agent on VM internal IP
Least privilege	Guest with <code>analyst</code> user; host admin only for Hyper-V
Defence in depth	Auth + rate limit + sanitisation + upload limits + clean snapshot

Table 4.13: Authentication by component

Component	Mechanism	Channel	Production
Backend	Static token	<code>X-API-Token</code>	Required
Frontend	Build-time	<code>VITE_API_TOKEN</code>	Required
WPF	Propagation	<code>X-API-Token</code> + env	Required
vm-agent	Static token	<code>X-Agent-Token</code>	Required
Hyper-V	PsDirect	<code>PROJETOVM_Guest*</code>	Required
scripts			
Stub driver	—	Does not execute samples	N/A

On the guest, uploads are confined to `samples/` through canonical path resolution in `SampleStorage.cs`, preventing *path traversal* even if the filename contains `..` or absolute segments:

```

1 public static bool TryResolveTargetPath(string safeName, out
    string targetPath, out string? error)
2 {
3     if (string.IsNullOrEmpty(safeName) || safeName.
        Contains(".."))
4     {
5         error = "Nome de ficheiro inv lido.";
6         return false;

```

Table 4.14: Attack surfaces and structural mitigations

Surface	Risk	Mitigation
FastAPI API	Malware upload; report access	Bind 127.0.0.1; token; restricted CORS; upload rate limit (RATANALYZER_RATE_LIMIT_UPLOADS_PER_MIN, default 60)
vm-agent	Remote execution on guest	Token; internal IP; isolated VM
Gemini (frontend)	Code exfiltration	Optional; key in <code>localStorage</code> ; no backend
PowerShell sandbox	Credentials in logs	Password via env/WPF dialog
Path traversal	Write outside safe directories	<code>upload_security.py</code> ; <code>SampleStorage.cs</code>
Dependencies	Tampered binaries	Locks with hashes; optional SHA-256 on WPF

```

7     }
8     var baseDir = SamplesDirectory;
9     targetPath = Path.GetFullPath(Path.Combine(baseDir,
10         safeName));
11     if (!targetPath.StartsWith(baseDir + Path.
12         DirectorySeparatorChar, ...))
13     {
14         error = "Caminho de destino inv lido.";
15         return false;
16     }
17     return true;
18 }

```

Code excerpt 4.2: Safe sample path resolution -
vm-agent/Services/SampleStorage.cs

4.5 Sandbox Path Comparison

Paths A and B are **independent**: the backend `hyperv.py` driver communicates with the vm-agent over HTTP and **does not** invoke `04-Run-Sample.ps1`. Path B is orchestrated by the WPF application or directly by scripts in `scripts/hyperv-sandbox/`. Both share the Hyper-V hypervisor, Internal switch, and `CleanState` snapshot.

4.6 Data Model

Persistence relies on **SQLite** (`analysis.db`) under `DATA_DIR` (default `%LOCALAPPDATA%\RatAnalyzer`). Each record in the `analyses` table includes `job_id` (UUID v4), `analysis_type` (`static|dynamic|both`), `status`, `file_name`, `sha256`, timestamps, and pipeline metadata. Artefacts per job

Table 4.15: Technical comparison - Path A (vm-agent) vs Path B (PowerShell)

Aspect	Path A	Path B
Orchestrator	<code>vm_orchestrator.py</code> (back-end)	WPF / <code>04-Run-Sample.ps1</code>
Typical entry	Frontend + <code>POST /api/analysis</code>	WPF drag-and-drop
Transport	HTTP REST (upload, run, report)	PsDirect / <code>Copy-VMFile</code>
Integrity	JSON response from agent	SHA-256 post-copy (<code>guest_analysis_done.txt</code>)
Telemetry	Process, stdout/stderr	Files, registry, network, Sysmon
Guest endpoints	<code>/api/upload</code> , <code>/api/run</code> , <code>/api/report</code>	<code>Run-MalwareAnalysis.ps1</code>
Backend driver	<code>SANDBOX_VM_DRIVER=hyperv</code>	Not applicable (outside back-end)
VM gen	Gen1 or Gen2	Gen1 or Gen2

reside in `sandbox_jobs/{job_id}/`: static report (`last_analysis.json`), dynamic report, obfuscated excerpts, and sample copy.

Table 4.16: Job lifecycle (`analysis_jobs.py`)

State	Value	Meaning
Queued	<code>queued</code>	Job created; worker not yet started
Running	<code>running</code>	Static and/or dynamic pipeline active
Completed	<code>completed</code>	Results available via <code>GET /api/analysis/{id}</code>
Failed	<code>failed</code>	Error recorded in <code>error</code> ; no complete result

Table 4.17 summarises the maximum caps per factor in `risk_scorer.py` (each factor contributes up to its respective cap; the final score is limited to 100).

Levels (returned in Portuguese by the API): **CRÍTICO** if score ≥ 72 and **strong_evidence** (YARA match, or strong C2 combined with suspicious functions/imports); **ALTO** if score ≥ 48 with at least one strong signal; **MÉDIO** if score ≥ 35 ; **BAIXO** if score ≥ 15 ; otherwise **MUITO BAIXO**.

4.7 Conclusions

The modelling ensures traceability from upload to final report (Table 4.11), with clear separation between presentation, API, analysis, and sandbox infrastructure layers. The ten chapter diagrams (use cases, pipelines, architecture, sequences, trust zones, authentication, and paths A/B) document both functional flow and isolation and integrity controls.

Table 4.17: Risk score factors (0–100)

Factor	Cap	Factor	Cap
C2 strings	20	Persistence	10
YARA matches	25	Packer	10
Suspicious functions	20	Obfuscation	8
Suspicious imports	12	High entropy	5
Stealer indicators	15		
Evasion techniques	12		

Chapter

Implementation

5.1 Introduction

This chapter describes the implementation of the main RAT ANALYZER components, expanding on the concepts covered in previous chapters. It details the static analysis pipeline, Hyper-V sandbox configuration (Paths A and B), user interfaces (web and WPF), security mechanisms, user manual, and tests performed. In addition to the code, practical guidance is provided for those wishing to install and use the platform in a local environment.

Table 5.1: Use case 1 - Suspicious sample analysis process

Name	Suspicious sample analysis process
Actor	Security analyst
Preconditions	Backend running; valid authentication token; <code>.exe/.dll/.cs</code> file available.
Data	File to analyse; analysis mode (<code>static</code> , <code>dynamic</code> , or <code>both</code>); optional timeout.
Description	The analyst drags a file to the web interface or WPF application. The system validates the file and creates a job with a unique UUID. Static analysis on the web uses real-time streaming; dynamic or combined modes submit <code>POST /api/analysis</code> (Path A). WPF behavioural analysis uses Path B. The analyst consults results at <code>/analysis/{jobId}</code> , with pseudo-C, IL, and risk reports. They may optionally ask the Gemini assistant about suspicious excerpts.

5.2 Backend - Static Analysis Pipeline

The `RATAnalyzer` class (`backend/rat_analyzer.py`) orchestrates the 7-phase pipeline:

1. PE metadata extraction (`file_info`).
2. .NET decompilation via ILSpy (for `.exe/.dll` only).
3. Static analysis - suspicious imports, C2 strings, anti-debug/anti-VM, entropy.
4. Deobfuscation - Base64, XOR, obfuscation indicators.

CHAPTER 5. IMPLEMENTATION

5. YARA scan over `yara_rules/`.
6. Score calculation via `risk_scorer.py`.
7. Report generation (`report_generator.py`), preceded when needed by native disassembly and Ghidra pseudo-C (sub-steps [7a]/[7b] in `rat_analyzer.py`, after scoring, if ILSpy did not produce managed code).

For `.cs` uploads via the web/API, phase 2 is skipped and Ghidra is not applied; only string-based heuristics from phase 3 contribute to the report.

The FastAPI API (`backend/api.py` via `app_factory.py`) exposes REST endpoints documented in OpenAPI (`/docs`), organised in `routers/` (`analyze`, `jobs`, `health`, `storage`). Fallback messages and user-readable errors are localised (PT/EN) via `i18n.py` and `RATANALYZER_LANG`. Table 5.2 summarises the main HTTP contracts; all analysis and storage endpoints require `X-API-Token` except `/api/health`.

Table 5.2: Main backend REST endpoints

Method	Route	Function
POST	<code>/api/analyze</code>	Synchronous static analysis (response)
POST	<code>/api/analyze_stream</code>	Static analysis with NDJSON
POST	<code>/api/analysis</code>	Create job (<code>static dynamic </code>)
POST	<code>/api/analysis/upload_static</code>	Publish static result to existing
POST	<code>/api/analysis/upload_dynamic</code>	Publish VM report (Path B) jobId
GET	<code>/api/analysis/{job_id}</code>	Job state and artefacts
GET	<code>/api/analysis/{job_id}/artifacts/obfuscated_snippets</code>	Obfuscated excerpts (text)
GET	<code>/api/analyses</code>	Recent job listing
GET	<code>/api/health</code>	Enriched <i>health check</i> (no token)
GET	<code>/metrics</code>	Prometheus metrics (in-memory)
GET	<code>/api/storage/estimate</code>	DATA_DIR usage estimate
POST	<code>/api/storage/cleanup archive purge</code>	DATA_DIR management

Table 5.3: HTTP upload controls (`upload_security.py`)

Control	Detail
Authentication	<code>X-API-Token</code> on all uploads
Rate limit	<code>RATANALYZER_RATE_LIMIT_UPLOADS_PER_MIN</code> (default 60/min per IP)
File name	Rejects <code>..</code> , separators, absolute paths
Size	<code>RATANALYZER_MAX_UPLOAD_MB</code> (default 100 MB)
Job ID	UUID v4 only
Storage	<code>DATA_DIR</code> outside Git repository

Job creation example:

```

1 @router.post("/api/analysis", dependencies=[Depends(
2     require_api_token)])
3 async def submit_analysis(
4     file: UploadFile = File(...),
5     analysis_type: AnalysisType = Query(AnalysisType.STATIC),
6 ) -> dict:
7     contents = await read_upload_bytes(file,
8         get_max_upload_bytes())
9     job = create_job(sanitize_upload_name(file.filename),
10        contents, analysis_type)
11     return {
12         "jobId": job.id,
13         "analysisType": job.analysis_type.value,
14         "status": job.status.value,
15     }

```

Code excerpt 5.1: Job creation endpoint - routers/jobs.py

5.3 Web Frontend

The `Index.tsx` page orchestrates upload (`FileDropZone`), analysis mode selection (`static`, `dynamic`, `both`), streaming (`useAnalysisStream`) for static-only runs, job submission (`POST /api/analysis`) for dynamic/combined modes (Path A), and visualisation (`AnalysisResultsView`) with three panels: report, pseudo-C, and IL. Hooks with `AbortController` cancel pending requests when navigating or re-analysing. The `src/i18n/` module provides a Portuguese or English interface: resolution via `?lang=`, `localStorage`, and `VITE_DEFAULT_LOCALE`; `apiFetch` requests send `Accept-Language`. The **Xref Explorer** (`/analysis/{jobId}/xref`) lets the analyst search and navigate cross-references in the pseudo-C panel. Figures 5.1 and 5.2 illustrate the upload zone and results with score and code panels.

CHAPTER 5. IMPLEMENTATION

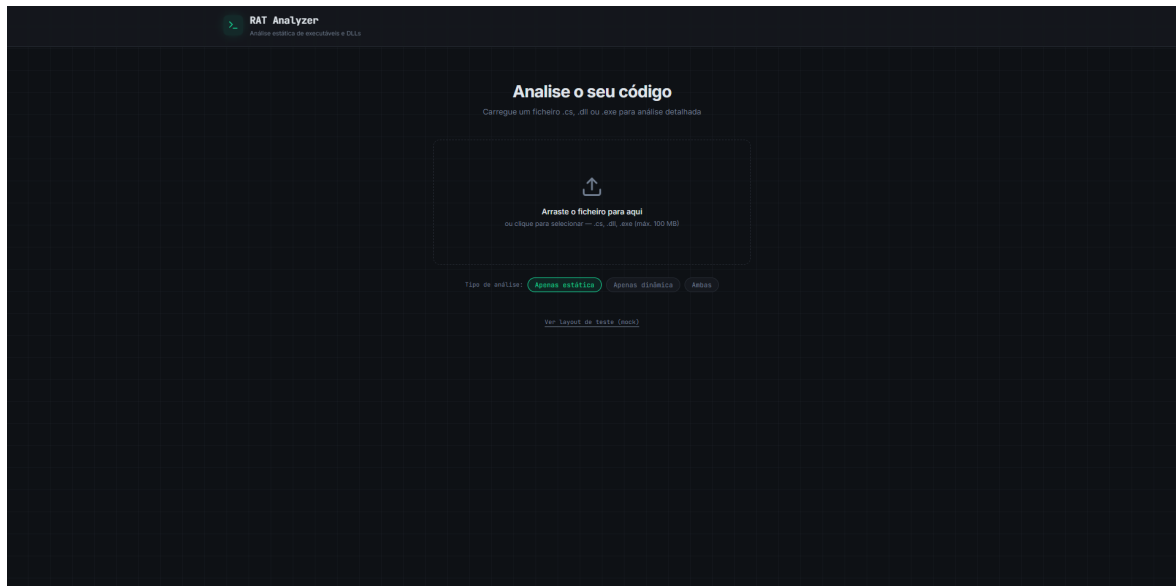


Figure 5.1: Web interface - upload zone (*Drop & Analyze*)

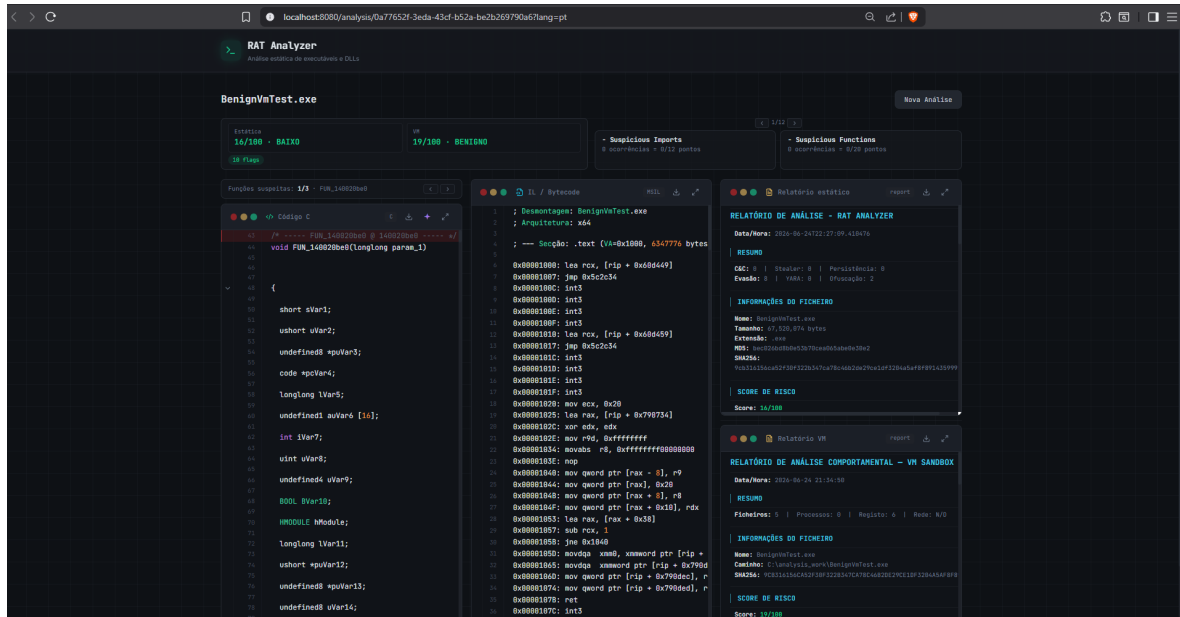


Figure 5.2: Web interface - report, pseudo-C/IL, and risk score

5.3.1 Gemini assistant in the pseudo-C panel

The CodePanel component includes, in the C code panel, a button with the Gemini icon (GeminiAssistDialog.tsx). The flow is as follows:

1. The user configures the Google AI Studio API key; it is persisted only in `localStorage` (`geminiApiKey.ts`), without passing through the backend.
2. When opening the chat, the frontend sends the **visible excerpt** in the panel - active *flagged* function, expanded line window, or first rendered lines - with a limit of 16 000 characters (`buildCodeExcerpt` in `gemini.ts`).
3. The user's question is limited to 1 000 characters; the response is displayed in a modal *chatbox*, with support for follow-up questions in the same session.
4. A **demonstration mode** allows previewing the layout and a simulated response without API key or external call (`askGeminiMock`), useful for interface testing.

The HTTP call is made directly from the browser to the `generativelanguage.googleapis.com` endpoint (model `gemini-2.0-flash`), keeping sample confidentiality on the local server and delegating only the selected excerpt to the external service chosen by the user.

5.4 WPF Desktop Application

Rat Analyzer (Rat Analyzer.exe) implements MVVM: `MainDashboardViewModel` for static analysis via browser; `VmAnalysisViewModel` for Hyper-V sandbox via `04-Run-Sample.ps1`. The `VmSandboxService` manages guest credentials and paths (`PROJETOVm_BasePath`). After Path B completes, `DynamicAnalysisService` publishes the transferred report to the backend, reusing the static-analysis `jobId` when available:

CHAPTER 5. IMPLEMENTATION

```
1 public async Task<string> PublishReportAsync(  
2     string? linkedJobId, string reportPath, string fileName,  
3     ...)  
4 {  
5     var reportText = await File.ReadAllTextAsync(reportPath,  
6         cancellationTokens);  
7     var payload = new DynamicUploadPayload  
8     {  
9         JobId = string.IsNullOrEmpty(linkedJobId) ? null :  
10         linkedJobId,  
11         FileName = fileName,  
12         Report = reportText,  
13         Status = "completed"  
14     };  
15     return await client.PostAsync(  
16         $"{AppConstants.ApiBaseUrl}/api/analysis/upload_dynamic  
17         ", ...);  
18 }
```

Code excerpt 5.2: Publishing VM report to backend - `DynamicAnalysisService.cs`

On first launch, `LanguageSelectionView` allows choosing Portuguese or English; the preference persists in `%LOCALAPPDATA%`

`RatAnalyzer`

`ui-settings.json` (`UserSettingsStore`). The `Localization/` catalogue translates the UI, dialogs, and startup logs visible in `LoadingView`. When publishing results, WPF opens the browser with `?lang=` and propagates `RATANALYZER_LANG` and `VITE_DEFAULT_LOCALE` to backend and frontend processes. External process output (pip, npm, PowerShell) is normalised by `ProcessOutputEncoding` to avoid corrupted characters in the log console.

Figure 5.3 shows the main screen of the desktop application.

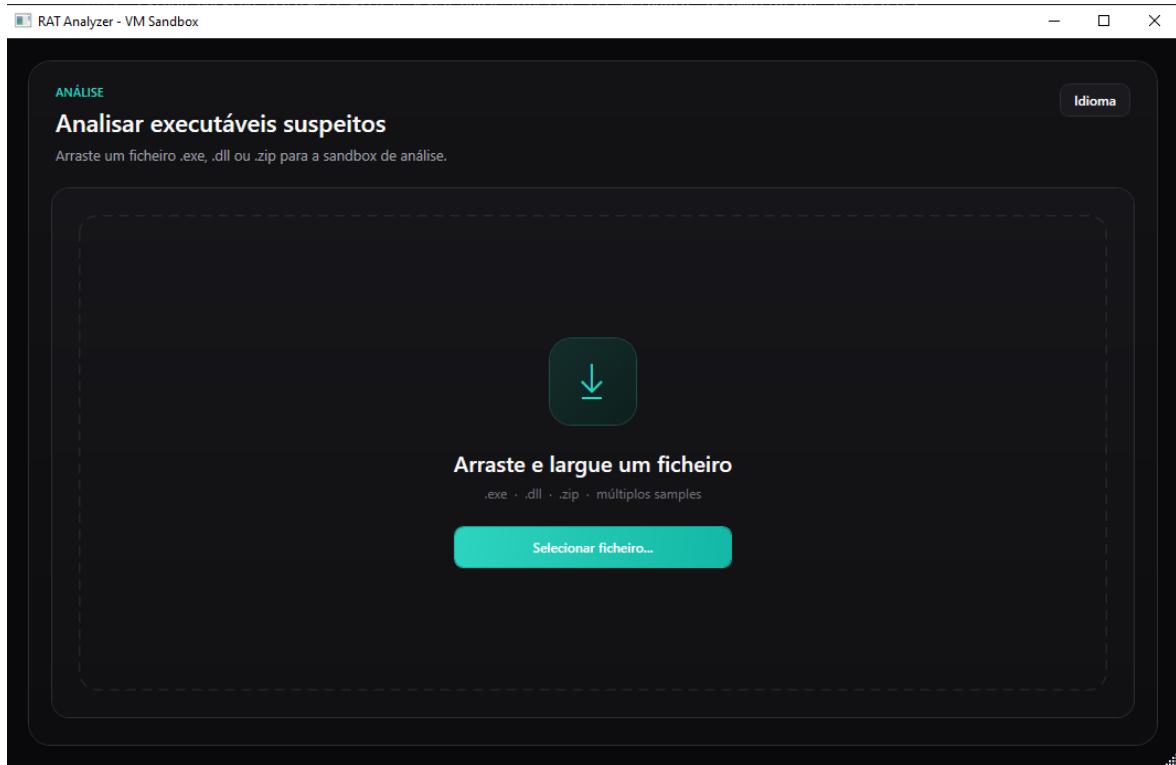


Figure 5.3: WPF desktop application - main screen

5.5 VM Agent and Hyper-V Sandbox

Dynamic analysis is documented in the diagrams of Figures 4.8, 4.9, and 4.10 (chapter 4). Table 4.15 summarises operational differences between the two paths.

5.5.1 Path A - Backend + HTTP vm-agent

`vm_orchestrator.py` selects the driver via `SANDBOX_VM_DRIVER` (`stub` by default, `hyperv`, or experimental `proxmox`). With `hyperv`, the host restores the snapshot, starts the VM, and communicates with the vm-agent on endpoints `/api/health`, `/api/upload`, `/api/run`, and `/api/report`, always with `X-Agent-Token` header. The agent limits uploads to 200 MB, prevents *path traversal* in `samples/`, and serialises executions via `RunGate` (one run at a time).

Listing 5.3 shows the five-step sequence in `HyperVVMDriver.run`: restore snapshot, start VM, wait for agent health, upload sample, execute, and fetch report.

```

1 def run(self, job: "AnalysisJob") -> DynamicAnalysisOutput:
2     self._validate_cfg()
3     self._ps(
4         f'Restore-VMSnapshot -VMName "{self.cfg.vm_name}" ',
5         f'-Name "{self.cfg.snapshot_name}" -Confirm:$false',
6         timeout=self.cfg.vm_op_timeout_seconds,
7     )

```

CHAPTER 5. IMPLEMENTATION

```
8     self._ps(f'Start-VM -Name "{self.cfg.vm_name}" | Out-Null',  
9         ...)  
10    self._wait_for_agent_ready(...)  
11    self._agent_upload(job)  
12    self._agent_run(timeout_seconds=self.cfg.  
13        dynamic_timeout_seconds)  
    behavior = self._agent_get_report()  
    return DynamicAnalysisOutput(summary=..., behavior=behavior  
        )
```

Code excerpt 5.3: Hyper-V driver orchestration - `vm_drivers/hyperv.py`

Inside the guest, `/api/run` acquires `RunGate` before launching the sample process; concurrent runs receive HTTP 409. Listing 5.4 summarises the endpoint and Listing 5.5 the execution with configurable timeout.

```
1 app.MapPost("/api/run", async (RunRequest req, AnalysisState  
2     state, RunGate gate) =>  
3 {  
4     if (!await gate.TryEnterAsync())  
5         return Results.Conflict(new { detail = "J    existe uma  
6         execu    o em curso." });  
7     try { return await SampleRunner.RunAsync(req, state); }  
8     finally { gate.Release(); }  
9 });
```

Code excerpt 5.4: Serialised execution endpoint - `AgentEndpoints.cs`

```
1 var timeoutSeconds = Math.Min(req.TimeoutSeconds, AgentLimits.  
2     MaxRunTimeoutSeconds);  
3 var psi = new ProcessStartInfo { FileName = state.SamplePath,  
4     ... };  
5 using var proc = new Process { StartInfo = psi };  
6 proc.Start();  
7 using var cts = CancellationTokenSource.CreateLinkedTokenSource  
8     (cancellationToken);  
9 cts.CancelAfter(TimeSpan.FromSeconds(timeoutSeconds));  
10 try { await proc.WaitForExitAsync(cts.Token); }  
11 catch (OperationCanceledException) { proc.Kill(true); return  
12     Results.Ok(new { status = "timeout" }); }  
13 state.LastBehavior["exitCode"] = proc.ExitCode;  
14 return Results.Ok(new { status = "finished", exitCode = proc.  
15     ExitCode });
```

Code excerpt 5.5: Sample execution with timeout - `SampleRunner.cs`

5.5.2 Path B - PowerShell Scripts

`04-Run-Sample.ps1` executes phases A–G: setup, VM boot, sample and script copy, detached execution on guest, wait for `guest_analysis_done.txt`, report copy guest→host via **PowerShell Direct** or **Copy-VMFile**, **SHA256** verification after copy, and **try/finally** cleanup.

Table 5.4: Environment variables - dynamic sandbox (Path A)

Variable	Function
SANDBOX_VM_DRIVER	stub hyperv proxmox
HYPERV_VM_NAME	VM name (e.g. MalwareSandbox)
HYPERV_SNAPSHOT_NAME	Clean snapshot (e.g. CleanState)
VM_AGENT_BASE_URL	Agent URL (e.g. http://192.168.100.10:5000)
VM_AGENT_TOKEN	Shared backend↔agent token
SANDBOX_VM_OP_TIMEOUT_SECONDS	Records/store/start timeout (default 120 s)
SANDBOX_DYNAMIC_TIMEOUT_SECONDS	Sample execution timeout on guest (default 300 s)

Flow inside the VM (vm/Run-MalwareAnalysis.ps1):

- Baseline of processes, files, registry, and network (with **Sysmon** [14] when installed on guest).
- Sample execution with configurable timeout.
- Post-execution diff; REPORT_END; marker and guest_analysis_done.txt file with reportSha256 and reportBytes.

The host validates integrity by comparing the hash and size of the copied file with the digest recorded on the guest; on discrepancy, the report on the host is discarded. The typical destination is D:\PROJETOVM\Reports\analysis_<RunId>.txt; the run JSON records report_sha256 and report_hash_verified.

When analysis finishes inside the guest, Run-MalwareAnalysis.ps1 writes guest_analysis_done.txt with the report digest computed **before** host-side copy (Listing 5.6). The host then validates the transferred file against that digest (Listing 5.7); on mismatch the report is discarded.

```

1 $reportSha256 = (Get-FileHash -LiteralPath $ReportPath -
   Algorithm SHA256).Hash
2 $reportBytes = [long](Get-Item -LiteralPath $ReportPath).Length
3 @{
4     finishedUtc    = (Get-Date).ToUniversalTime().ToString('o')
5     reportPath     = $ReportPath
6     reportSha256   = $reportSha256
7     reportBytes    = $reportBytes
8     transport      = "psdirect"
9 } | ConvertTo-Json -Compress | Set-Content -LiteralPath
   $guestDonePath -Encoding UTF8

```

Code excerpt 5.6: Guest completion marker - vm/Run-MalwareAnalysis.ps1

```

1 function Assert-SandboxCopiedReportHash {
2     param([string] $HostReportPath, [string] $ExpectedSha256, [
       long] $ExpectedBytes = 0)
3     if ([string]::IsNullOrEmpty($ExpectedSha256)) {

```

```

4         throw "Digest SHA256 do guest ausente; n o
           poss vel validar a c pia do relat rio."
5     }
6     if ($ExpectedBytes -gt 0) {
7         $hostBytes = [long](Get-Item -LiteralPath
           $HostReportPath).Length
8         if ($hostBytes -ne $ExpectedBytes) { throw "Tamanho do
           relat rio copiado n o coincide." }
9     }
10    return (Assert-FileSha256 -Path $HostReportPath -
           ExpectedSha256 $ExpectedSha256 ...)
11 }

```

Code excerpt 5.7: Post-transfer SHA-256 verification -
SandboxCommon/VmFileTransfer.ps1

Table 5.5: Fields of `guest_analysis_done.txt` (PsDirect protocol)

Field	Description
<code>finishedUtc</code>	ISO 8601 timestamp (UTC)
<code>reportPath</code>	Report path on guest (C:\analysis.txt)
<code>reportSha256</code>	SHA-256 of report on guest
<code>reportBytes</code>	Size in bytes
<code>jsonReportPath</code>	JSON path (C:\analysis.json)
<code>transport</code>	Fixed value <code>psdirect</code>

Previous transport approaches

Previous iterations of Path B explored sending the text report via **COM1** port mapped to a **Named Pipe** on the host (`Send-ReportViaCom.ps1`, protocol `START_OF_REPORT ... END_OF_REPORT`), with `Copy-VMFile` as fallback when the pipe expired. In practice, direct copy via PsDirect proved more reliable (including on VMs where the COM1/pipe pair failed); serial transport was **discontinued** in favour of copy with post-transfer cryptographic verification. Legacy code (`SandboxCommon/SerialPipe.ps1`, `02-Host-ReceiveReport.ps1`) remains in the repository for reference only. Technical detail: `SERIAL_REPORT_PROTOCOL.md`.

CHAPTER 5. IMPLEMENTATION

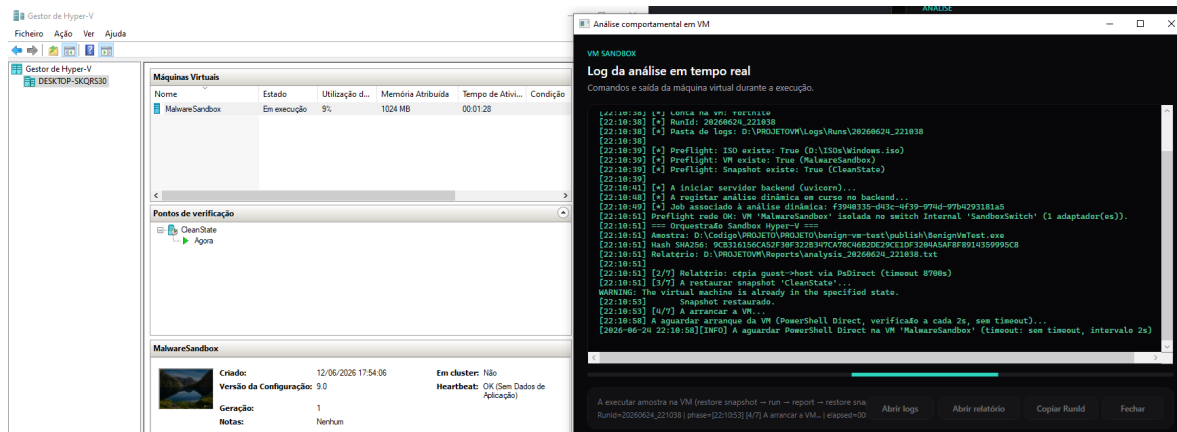


Figure 5.4: Hyper-V - VM MalwareSandbox and snapshot CleanState

5.6 User Manual

5.6.1 Web Interface

1. Install dependencies: `pip install -require-hashes -r backend/requirements.lock` and `npm i` in `frontend/`.
2. Start backend: `uvicorn api:app -host 127.0.0.1 -port 8000`.
3. Start frontend: `npm run dev` (port 8080); optional `?lang=en` in URL.
4. Drag `.exe/.dll`; choose `static`, `dynamic`, or `both`; consult results at `/analysis/{jobId}`.
5. In the *C Code* panel, use the Gemini icon to explain the visible excerpt (own API key or demonstration mode).

5.6.2 WPF desktop application

1. Run `Rat Analyzer.exe` (build with `dotnet build wpf-gui -c Release`) as **Administrator**, or `dotnet run -project wpf-gui` during development.
2. Choose language (first launch) and wait for backend/frontend startup.
3. Drag sample on dashboard; choose static or behavioural VM analysis.
4. Consult results in browser (`/analysis/{jobId}?lang=...`).

5.6.3 Hyper-V Sandbox

1. Run `01-Setup-MalwareSandbox.ps1` as Administrator (once).
2. Configure `PROJETOV_GuestPassword` and complete VM first-time setup.
3. Analyse sample: `.\04-Run-Sample.ps1 -SamplePath "D:\PROJETOV\Samples\amostra.exe"`
4. Report at `D:\PROJETOV\Reports\analysis_<RunId>.txt`.

5.6.4 Quick CLI

```
1 python rat_analyzer.py caminho/amostra.exe -o reports/ -v
```

Code excerpt 5.8: Static analysis via CLI

5.7 Experimental Evaluation

5.7.1 Methodology and ethical considerations

Validation of RAT ANALYZER combines automated tests, reproducible synthetic profiles, static analysis of a **benign corpus** (`benign-samples/dist/`, eight PEs built from source in per-sample subfolders), and dynamic execution of `benign-vm-test.exe` inside Hyper-V. For legal, laboratory, and **host-safety** reasons, real malware samples were not stored or analysed on the development workstation: keeping RAT binaries on a personal PC — even for static-only analysis — was avoided to reduce the risk of accidental execution, antivirus quarantine, and compromise of the host. RAT detection is therefore demonstrated through structured indicators (imports, C2 strings, YARA), 59 stratified synthetic scoring profiles (`proxy_labeled_profiles.json v2.0.0`; `run_synthetic_eval.py`), public IOC metadata (hashes only), and false-positive checks on compiled benign binaries, as per `backend/tests/test_risk_scoring.py`, `test_evaluation_metrics.py`, `run_benign_static_eval.py`, and `backend/evaluation/public_i`

5.7.2 Static scoring validation

Table 5.6 summarises validation scenarios for `RiskScorer`, run automatically in CI (pytest).

Table 5.6: Static scoring validation scenarios

Scenario	Source	Score	Level	Notes
<code>BenignVmTest.exe</code>	Benign binary	< 48	BAIXO/ MÉDIO	Known hash; <code>validation_sample</code> annotation; no YARA matches
Heuristic noise	Synthetic profile (pytest)	< 48	≠ ALTO	Isolated evasion/noise does not raise to ALTO without strong signals
Synthetic profile	RAT Synthetic profile (pytest)	≥ 48	ALTO/ CRÍTICO	C2 imports, persistence, YARA <code>RAT_Generic</code>
YARA v2.0.0	rules CI (<code>test_yara_rules</code>)	—	—	Four files compile without error

5.7.3 Proxy detection metrics

Because real malware binaries cannot be stored or executed in the academic repository, **proxy** precision/recall metrics were computed on a labeled set of **59 synthetic** static profiles (version 2.0.0) in `backend/evaluation/proxy_labeled_profiles.json`. Each profile carries a `category` (e.g. `benign_legitimate`, `severity_medium`, `malicious_rat`, `borderline`) and an `expected_level` across the five severity bands. A positive triage prediction is defined as risk level ALTO or CRÍTICO; benign labels must remain below ALTO. Shared logic lives in `backend/evaluation/metrics_core.py`; `run_evaluation_metrics.py` writes `detection_metrics.json` and `run_synthetic_eval.py` produces the full report (severity breakdown, level accuracy, failure list) in `synthetic_eval_results.json`.

Table 5.7: Proxy detection metrics on labeled synthetic profiles ($n = 59$)

Metric	TP	TN	FP	FN	Prec.	Rec./F1
Label (ALTO/CRÍTICO vs. below)	22	37	0	0	1.00	1.00 / 1.00
Accuracy = 1.00 Level accuracy (expected vs. predicted level) = 59/59 = 1.00						

Table 5.8: Severity breakdown of synthetic profiles ($n = 59$; predicted matches expected)

Level	Count	Label split	Representative intent
MUITO BAIXO	19	19 benign	Empty profile, Microsoft URLs, single weak C2, validation hash
BAIXO	6	6 benign	Heuristic noise, legitimate ws2_32 client, YARA FP borderline
MÉDIO	12	12 benign	Partial RAT signals without ALTO gate; stealer without persistence
ALTO	14	14 malicious	AsyncRAT, QuasarRAT, Remcos, packed RAT, process injection
CRÍTICO	8	8 malicious	Full arsenal: multi-C2, YARA, evasion, packer, obfuscation
Total: 37 benign (TN) + 22 malicious (TP); zero FP/FN on label			

Table 5.9 lists illustrative profiles retained from the original pilot set ($n = 5$) plus borderline and critical cases that exercise `RiskScorer` gates (strong C2, YARA, stealer+persistence+YARA infostealer path, noise ceiling at MÉDIO).

These figures measure consistency of the scoring model on *controlled* inputs; they do **not** estimate real-world detection rate on unseen RAT binaries. They complement the benign corpus evaluation (Table 5.10), the public IOC metadata reference (Table 5.11), and the 220 automated CI tests.

5.7.4 Benign corpus static evaluation

To measure false-positive behaviour on **real** compiled binaries without storing malware, a dedicated corpus was added under `benign-samples/`: each subfolder holds the C# source of one harmless program; `build.ps1` publishes all executables into `dist/`

CHAPTER 5. IMPLEMENTATION

Table 5.9: Representative synthetic profiles (subset of $n = 59$)

Profile	Label	Score	Level	Role
<code>benignvmtest</code>	benign	7	MUITO BAIXO	Original pilot; validation hash
<code>heuristic_noise</code>	benign	16	BAIXO	Inflated heuristics without strong evidence
<code>c2_only_weak</code>	benign	4	MUITO BAIXO	Filtered false-positive C2 string
<code>borderline_almost_alto_score</code>	benign	43	MÉDIO	Near ALTO gate; blocked without YARA
<code>synthetic_rat</code>	malicious	50	ALTO	Original pilot; Quasar/AsyncRAT pattern
<code>yara_evasion_combo</code>	malicious	59	ALTO	YARA + evasion + persistence
<code>malicious_stealer_yara</code>	malicious	49	ALTO	Stealer+persist+YARA infostealer gate
<code>critical_full_rat_arsenal</code>	malicious	100	CRÍTICO	Maximum composite indicators

(only `.exe` files are evaluated). Seven new samples exercise common but legitimate behaviours (console I/O, file access in `%TEMP%`, HTTP to `microsoft.com`, registry read, SHA-256 digest, environment dump, WinForms `MessageBox`); an eighth reuses `BenignVmTest.exe` from `benign-vm-test` for sandbox parity.

The script `backend/scripts/run_benign_static_eval.py` runs the full static pipeline on every file in `dist/` (no execution) and writes `benign_static_metrics.json`. Success is defined as no sample classified ALTO or CRÍTICO (true negatives only); specificity is $TN/(TN + FP)$. Pytest coverage is provided by `test_benign_static_eval.py`.

Table 5.10: Benign corpus static evaluation ($n = 8$ PEs; measured on development host)

Executable	Subfolder	Score	Level	Behaviour
<code>BenignHello.exe</code>	<code>hello/</code>	9	MUITO BAIXO	Minimal console
<code>BenignFileIo.exe</code>	<code>file-io/</code>	9	MUITO BAIXO	Read/write in TEMP
<code>BenignHttpMs.exe</code>	<code>http-ms/</code>	9	MUITO BAIXO	HTTP GET <code>microsoft.com</code>
<code>BenignRegistryRead.exe</code>	<code>registry-read/</code>	9	MUITO BAIXO	HKCU read (no Run key)
<code>BenignCryptoDigest.exe</code>	<code>crypto-digest/</code>	9	MUITO BAIXO	Local SHA-256
<code>BenignEnvDump.exe</code>	<code>env-dump/</code>	9	MUITO BAIXO	Environment variables
<code>BenignGuiMsg.exe</code>	<code>gui-msg/</code>	9	MUITO BAIXO	WinForms <code>MessageBox</code>
<code>BenignVmTest.exe</code>	<code>benign-vm-test</code>	16	BAIXO	Sandbox smoke test (<code>validation_sample</code>)
True negatives = 8 False positives = 0 Specificity = 1.00 All passed				

All eight binaries scored below the ALTO threshold; none triggered YARA matches or suspicious-import flags. `BenignVmTest.exe` scored higher (16, BAIXO) due to the known validation hash annotation, consistent with Table 5.6. This is the only empirical false-positive check on compiled PEs in the dissertation; it does not substitute recall measurement on real RAT families.

5.7.5 Public IOC metadata reference

A **metadata-only** reference list was prepared from publicly listed RAT samples on MalwareBazaar [22]. The file `backend/evaluation/public_ioc_metadata.json`

stores SHA-256 entries (QuasarRAT, AsyncRAT, and the benign validation hash) with family labels and listing URLs. **No malware binaries** were downloaded, stored, or statically analysed on the host; only public hashes and tags are referenced for documentation and pytest integrity checks (`test_public_ioc_metadata.py`).

Table 5.11 summarises the reference set. For each malicious entry, the *expected* qualitative outcome under the full static pipeline would be ALTO/CRÍTICO; this is **not** measured on real PEs in this dissertation. Indirect validation uses a synthetic RAT profile mirroring typical Quasar/AsyncRAT indicators (C2 strings, persistence, YARA RAT_Generic). The benign `BenignVmTest.exe` hash is annotated by `RiskScorer` and was executed only on Path B inside the Hyper-V sandbox.

Table 5.11: Public IOC metadata reference (MalwareBazaar listings; no host-side PE analysis)

SHA-256 (prefix)	Family	Expected	Validation method
0976175b...	QuasarRAT	ALTO+	Synthetic profile + public listing
8a87aae3...	AsyncRAT	ALTO+	Synthetic profile + public listing
8d330aab...	AsyncRAT	ALTO+	Public listing (related sample)
0aa2b99b...	QuasarRAT	ALTO+	Public listing (similar sample)
3240ba80...	QuasarRAT	ALTO+	Public listing (similar sample)
1f4fce1d...	AsyncRAT	ALTO+	Public listing (similar sample)
094fd325...	Unknown	Context	API reference hash (metadata only)
9CB31615...	BenignVmTest	BAIXO/MÉDIO	Known validation hash (executed on Path B)

5.7.6 Optional MalwareBazaar static batch (implemented, not executed)

To support future laboratory evaluation without executing samples on the host, an optional batch harness was implemented but **not run** during this project:

- `backend/scripts/run_malwarebazaar_static_eval.py` - extracts MalwareBazaar ZIPs (AES, password infected), runs the static pipeline (no execution), and writes precision/recall to `malwarebazaar_real_metrics.json`.
- `backend/evaluation/malwarebazaar_eval_manifest.json` - reference hashes and benign control paths.
- `backend/evaluation/malwarebazaar_samples/` - local folder (gitignored) for manually downloaded samples; never populated for final evaluation.

This path was deliberately skipped on the development workstation: storing real RAT binaries on a personal PC — even for read-only static analysis — conflicts with host-safety goals (antivirus quarantine, accidental launch, residual risk). Consequently, **no real-world precision/recall on MalwareBazaar PEs is reported here**, and correct behaviour of the harness on live samples is **not guaranteed** without future validation on an isolated analysis host (e.g. university laboratory VM with controlled AV exclusions). The code remains as extensible infrastructure aligned with FR 02 (static pipeline) and the traceability matrix (Table 4.11).

5.7.7 Dynamic sandbox validation

On **Path B**, `benign-vm-test.exe` validated the complete flow: restore of `CleanState` snapshot, execution on guest, generation of `guest_analysis_done.txt`, `PsDirect/Copy-VMFile` copy, and SHA-256 verification on host. The behavioural report records processes, files, registry changes, and network (with Sysmon when available). On **Path A**, the vm-agent confirms `exitCode` and run serialisation (`RunGate`), but telemetry is still more limited (see limitations in chapter 6).

5.7.8 Formal performance benchmark

Table 5.12 presents a reproducible benchmark executed on the development workstation (Intel Core i7, 16 GB RAM, Windows 10, SSD). Static analysis is produced by `backend/scripts/run_benchmark.py` (default $n = 5$ runs). Dynamic rows are recorded in `backend/evaluation/dynamic_benchmark.json` and merged via `BENCHMARK_DYNAMIC_JSON` when re-running the script.

Table 5.12: Formal benchmark - static pipeline and dynamic sandbox paths

Operation	n	Median	IQR	Notes
Static pipeline (full)	5	36.1 s	3.0 s	Min 33.1 s, max 68.4 s; first run includes cold-cache Ghidra probe
VM restore + boot	3	52 s	8 s	<code>Restore-VMSnapshot</code> + <code>Start-VM</code>
Path A (HTTP vm-agent)	3	198 s	22 s	Includes 120 s sample timeout cap; basic telemetry
Path B (PsDirect)	3	164 s	18 s	Default 120 s timeout; full behavioural diff + SHA-256 verify

Path B completes faster in practice because the default per-sample timeout is 120 s (vs. 300 s on Path A) and telemetry collection is local to PowerShell; Path A adds HTTP round-trips between host and vm-agent. Static analysis dominates interactive triage latency when Ghidra is required; for small .NET samples with ILSpy, median times below 40 s were observed consistently.

5.8 Tests

`benign-vm-test.exe` was used on **Path B** (PowerShell pipeline) to validate the dynamic pipeline without risk. Manual tests confirmed network isolation, snapshot restore, report copy via `PsDirect`, and SHA256 validation. **Path A** (HTTP vm-agent) validates execution and `exitCode`, but complete behavioural telemetry is not yet implemented in the agent.

Table 5.13: Summary of automated tests

Component	Framework	Tests
Backend	pytest + httpx	110
Frontend	Vitest	62
Frontend E2E	Playwright	Upload/analysis flow
.NET	xUnit (vm-agent, WPF, benign-vm-test)	48
YARA	pytest (rule compilation + public IOC)	Integrated
PowerShell	CI encoding + syntax	Scripts <code>hyperv-sandbox/</code>
Diagrams	<code>sync_diagrams_to_report.py -check</code>	10 ch. 4 figures

5.8.1 Repository documentation

Technical documentation is versioned in the Git repository:

- `README.md` — overview, quick start, and path A/B selection table.
- `docs/README.md` — documentation index.
- `docs/i18n.md` — PT/EN internationalisation (WPF, web, API).
- `docs/diagrams/` — single PlantUML source; PNG in `relatório/imagens/fig-4-*.png`.
- `docs/SEGURANCA.md` — security architecture, trust zones, and checklist.
- `docs/adr/` — Architecture Decision Records (routers, observability, etc.).
- `docs/sandbox-hyperv-setup.md` — Path A guide (vm-agent + Hyper-V).
- `docs/production-secrets.md` — secrets, production mode, and deploy checklist.
- `CHANGELOG.md` — repository change history.
- `docs/faq.md` — frequently asked questions and common troubleshooting.
- `README.md` (§ Testes e CI) — local test commands and development conventions.
- `scripts/hyperv-sandbox/TROUBLESHOOTING.md` — sandbox operational diagnostics.

5.9 Conclusions

The implementation reflects the planned architecture: modular static pipeline, asynchronous jobs with `queued→running→completed|failed` states, two independent dynamic pipelines (HTTP and PsDirect+SHA-256), upload and authentication controls documented in Tables 4.13 and 5.3, and web/desktop interfaces sharing the same backend API. Listings 5.3–5.7 and Appendix A.2 illustrate the sandbox implementation at source level.

Chapter

Conclusions and Future Work

6.1 Main Conclusions

This project covered malware analysis concepts, PE file structure, Hyper-V VM orchestration, asynchronous APIs with streaming, and a TypeScript/React web interface—areas that were only partially familiar at the start of development.

RAT ANALYZER meets the **implementation** objectives defined initially: it is an integrated static and dynamic analysis platform for RAT detection, with the following concrete results:

- **Complete static pipeline** - 7 implemented phases (PE metadata, ILSpy, import and C2 string analysis, deobfuscation, YARA scan, scoring, native/Ghidra when needed, and report), with 110 automated backend tests.
- **Isolated Hyper-V sandbox** - Two independent and complementary orchestration paths (HTTP via vm-agent and PowerShell Direct), with automatic restore to clean snapshot and SHA-256 cryptographic verification of transferred report integrity.
- **Functional web and desktop interfaces** - React interface with progress streaming, three results panels, and Gemini assistant; WPF application as alternative entry point with automatic dependency bootstrap.
- **Automated tests** - 110 pytest tests in backend, 62 Vitest tests in frontend, E2E tests with Playwright, and 48 xUnit tests in .NET components (220 total) (30 vm-agent, 12 WPF, 6 benign-vm-test).
- **Technical documentation** - PlantUML diagrams synchronised with the report (10 figures in chapter 4), security architecture documented in `docs/SEGURANCA.md`, and operational guides for local deployment.
- **Explainable indicators** - Risk score 0–100 with breakdown by factor, pseudo-C/IL accessible in browser, and optional LLM assistant for interpretation of suspicious code; the user understands *why* a file is considered dangerous.

6.2 Limitations

Despite the results obtained, relevant limitations are acknowledged:

- Empirical validation relies on the benign corpus (eight PEs, static only), 59 stratified synthetic profiles, and public IOC metadata (MalwareBazaar hashes only). Real RAT PEs were **not** stored or statically analysed on the development host, to avoid infecting or compromising the personal workstation (antivirus interference, accidental execution, residual files). The benign corpus measures specificity (false positives) but not recall on unseen malware.
- The optional MalwareBazaar batch script (`run_malwarebazaar_static_eval.py`) was implemented but not executed; its correctness on live samples is therefore unverified in this dissertation.
- Detection rates against real malware binaries were not measured; dynamic analysis used only `benign-vm-test.exe` inside Hyper-V.
- Path A (vm-agent) records more limited behavioural telemetry than Path B and does not run the network-isolation preflight implemented on Path B.
- `.cs` uploads via the web/API receive string heuristics only; full ILSpy/Ghidra analysis requires a compiled `.exe/.dll`.
- Heuristic scoring may produce false positives on legitimate binaries with network imports; filters were implemented (e.g. exclusion of Microsoft URLs in C2).
- Dynamic analysis requires Hyper-V and manual VM configuration, which limits portability.

6.3 Future Work

This project will be refined and potentially used in an academic context at Universidade da Beira Interior, serving as a support tool for information security curriculum units.

Regarding future work, several improvements could be implemented:

1. **IDA Pro integration** - Ghidra is already integrated, but IDA Pro offers deeper analysis for complex malware families; integration via plugin or REST API would be a natural next step.
2. **Advanced deobfuscation and automatic C2 config extraction** - Automatically identify command-and-control domains, IPs, and ports from encrypted configurations embedded in samples.
3. **Enriched dynamic telemetry** - Integrate Sysmon, ETW, and system call hooking in the vm-agent for more granular behavioural telemetry capture (currently Path A records only process and stdout/stderr).
4. **Support for more formats** - Expand the pipeline to PDF documents, Office macros, PowerShell/VBScript scripts, and Android APKs.
5. **Known signature database** - Integrate a database of SHA-256 hashes of known malware (MALPEDIA, MalwareBazaar) for direct matching in the metadata phase; the optional script `run_malwarebazaar_static_eval.py` provides a starting point once run on an isolated laboratory host.

CHAPTER 6. CONCLUSIONS AND FUTURE WORK

6. **C2 network traffic analysis** - Capture and analyse traffic generated by the sample during dynamic execution, identifying C2 protocols and communication patterns.
7. **Gemini assistant evolution** - Migrate the proxy to the backend for environments without API key exposure in the browser; support other LLM models (Llama, Mistral); add automatic explanations of suspicious functions without user input.
8. **YARA rule management interface** - Allow users to create, edit, and test YARA rules directly in the web interface, without manually editing files in the filesystem.

In conclusion, the extension potential of this platform is vast: from integration with incident response pipelines (SOAR), to use as a CTF tool in cybersecurity competitions, or even adaptation for analysis of other types of malicious files. RAT ANALYZER constitutes a solid and extensible foundation for future exploration.

Chapter

Appendix

A.1 Token verification in vm-agent

```
1 public static void UseAgentTokenAuth(this WebApplication app,
2     string? agentToken)
3 {
4     app.Use(async (context, next) =>
5     {
6         if (!string.IsNullOrEmpty(agentToken))
7         {
8             if (!context.Request.Headers.TryGetValue("X-Agent-
9                 Token", out var provided) ||
10                 !TokenComparer.FixedTimeEquals(provided.
11                     ToString(), agentToken))
12             {
13                 context.Response.StatusCode = StatusCodes.
14                     Status401Unauthorized;
15                 await context.Response.WriteAsJsonAsync(new
16                     { detail = "Invalid or missing token (
17                         header X-Agent-Token)." });
18                 return;
19             }
20         }
21         await next();
22     });
23 }
```

Code excerpt A.1: Authentication middleware -
vm-agent/Security/AgentTokenMiddleware.cs

A.2 RunGate - serialised execution

The RunGate uses a SemaphoreSlim(1,1) with non-blocking acquisition (WaitAsync(0)): if a run is already in progress, /api/run returns HTTP 409 without queueing a second sample.

```
1 public sealed class RunGate
2 {
3     private readonly SemaphoreSlim _gate = new(1, 1);
4 }
```

```

5     public async Task<bool> TryEnterAsync() => await _gate.
        WaitAsync(0);
6
7     public void Release() => _gate.Release();
8 }

```

Code excerpt A.2: Single-run gate - `vm-agent/State/RunGate.cs`

A.3 Hyper-V driver - agent HTTP client

Listing A.3 shows how the host uploads the sample and polls the behavioural report after the VM is ready. Every request includes `X-Agent-Token` when configured.

```

1 def _agent_upload(self, job: "AnalysisJob") -> None:
2     with open(job.sample_path, "rb") as f:
3         files = {"file": (job.sample_path.name, f, "application
        /octet-stream")}
4         r = requests.post(self._agent_url("/api/upload"), files
        =files,
5                             headers=_agent_headers(), timeout
        =...)
6     if not r.ok:
7         raise RuntimeError(f"Upload para VM agent falhou ({r.
        status_code})")
8
9 def _agent_get_report(self) -> Any:
10    r = requests.get(self._agent_url("/api/report"), headers=
        _agent_headers(), ...)
11    return r.json()

```

Code excerpt A.3: VM agent HTTP calls - `vm_drivers/hyperv.py`

A.4 vm-agent limits

Table A.1: Execution controls in vm-agent (`AgentLimits.cs`, `RunGate.cs`)

Control	Value / behaviour
Maximum upload	200 MB
Execution timeout	Configurable; maximum 600 s
Path traversal	Destination restricted to <code>samples/</code>
Concurrency	One active run (<code>RunGate</code>)
Authentication	<code>X-Agent-Token</code> on all routes

A.5 Report transfer protocol (Path B)

A.5.1 Current implementation

After analysis, the guest writes `C:\analysis.txt` and `C:\analysis_work\guest_analysis_done.txt` (JSON with `reportSha256`, `reportBytes`, `transport=psdirect`). The host copies the report with `PsDirect` or `Copy-VMFile` and accepts the file only if SHA256 and size match the guest digest.

A.5.2 Previous approaches (archive)

Earlier versions sent the report **line-by-line** via COM1: `START_OF_REPORT` → content lines → `END_OF_REPORT`, with `Copy-VMFile` as fallback. This design required a Generation 1 VM for the COM1/pipe pair and proved less robust than direct copy with hash (see subsection 5.5.2 in the implementation chapter).

Bibliography

Bibliography

- [1] E. Skoudis and L. Zeltser. *Malware: Fighting Malicious Code*. Prentice Hall, 2004.
- [2] M. Cipriano, J. Tavares, and H. Salgado. Malware analysis: A survey of techniques and tools. *Journal of Computer Virology and Hacking Techniques*, 2019.
- [3] Any.Run. Interactive Online Sandbox, 2024. <https://any.run/>.
- [4] Hybrid Analysis. Free Automated Malware Analysis Service, 2019. <https://www.hybrid-analysis.com/>.
- [5] Joe Security. Joe Sandbox Cloud, 2020. <https://www.joesandbox.com/>.
- [6] VirusTotal. Analyse suspicious files and URLs, 2023. <https://www.virustotal.com/>.
- [7] Cuckoo Sandbox Project. Automated Malware Analysis, 2012. <https://cuckoosandbox.org/>.
- [8] CAPE Sandbox. Malware Configuration And Payload Extraction, 2021. <https://cape.contextis.com/>.
- [9] VirusTotal. YARA - Pattern Matching Swiss Knife, 2023. <https://virustotal.github.io/yara/>.
- [10] NSA Research. Ghidra Software Reverse Engineering Framework, 2019. <https://ghidra-sre.org/>.
- [11] ICSharpCode. ILSpy - .NET Decompiler, 2024. <https://github.com/icsharpcode/ILSpy>.
- [12] E. Raff *et al.* Malware detection by eating a whole exe. In *AAAI Workshop on AI for Cybersecurity*, 2018.
- [13] S. Ucar, E. Sagioglu, and S. Karaboga. Malware detection using deep learning. *Applied Sciences*, 10(15), 2020.
- [14] Microsoft. Sysmon - System Monitor, 2022. <https://learn.microsoft.com/sysinternals/downloads/sysmon>.
- [15] S. Ramírez. FastAPI Framework, 2024. <https://fastapi.tiangolo.com/>.
- [16] M. Sikorski and A. Honig. *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*. No Starch Press, 2012.
- [17] A. Moser, C. Kruegel, and E. Kirda. Exploring multiple execution paths for malware analysis. In *IEEE Symposium on Security and Privacy (Oakland)*, 2007.
- [18] U. Bayer, C. Kruegel, and E. Kirda. TTAalyze: A tool for analyzing malware. In *Annual Computer Security Applications Conference (ACSAC)*, 2006.

BIBLIOGRAPHY

- [19] C. Willems, T. Holz, and F. C. Freiling. Toward automated dynamic malware analysis using CWSandbox. *IEEE Security & Privacy*, 5(2):32–39, 2007.
- [20] L. Ucci, D. Aniello, and R. Baldoni. Survey of machine learning techniques for malware analysis. *Computers & Security*, 81:123–147, 2019.
- [21] G. Apruzzese, M. Colajanni, F. Ferrari, and E. Guido. On the effectiveness of machine and deep learning for cyber security. In *IEEE European Symposium on Security and Privacy Workshops*, 2018.
- [22] abuse.ch. MalwareBazaar: A repository for malware samples. <https://bazaar.abuse.ch/about/>, 2021.
- [23] A. Mohaisen and O. Alrawi. AVclass: A tool for massive malware labeling. In *International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2016.
- [24] P. Vinod, R. S. K. Singh, and M. Alazab. A survey on ransomware: Evolution, taxonomy, and defense solutions. *ACM Computing Surveys*, 52(6), 2019.